

Deep Learning: CNN

Vladimir I N Bykov

Agenda

- CNN
- typer av neurala nätverk för bildanalys, bildförberedelse och analys
- MNIST datasetet
- FER-2013 datasetet

Vad är Bildanalys?

Bildanalys är processen att extrahera meningsfull information från bilder genom användning av olika tekniker, algoritmer och modeller. Det kan vara allt från att känna igen objekt i en bild till att analysera pixeldata för att hitta mönster eller segmentera olika delar av bilden.

Tillämpningar:

Medicin: Bildanalys används i radiologi, för att analysera röntgenbilder, MRI-skanningar och andra typer av medicinska bilder för att upptäcka sjukdomar som tumörer, frakturer, eller avvikelser.

Säkerhet: Används i övervakning och ansiktsigenkänning för att öka säkerheten på offentliga platser.

Fordonsindustrin: Självkörande bilar använder bildanalys för att förstå sin omgivning, detektera fotgängare, fordon och trafikskyltar.

Jordbruk: Analysera satellitbilder för att övervaka grödor och upptäcka skadedjur eller sjukdomar.

Sociala Medier: Plattformer som Facebook använder bildanalys för att identifiera personer i bilder och ge rekommendationer om taggning.

Nyckelbegrepp:

Objektigenkänning: Processen att identifiera och lokalisera objekt i en bild, till exempel att känna igen en bil, ett ansikte eller en skylt.

Bildklassificering: Att tilldela en bild en viss kategori baserat på innehållet, till exempel att avgöra om en bild innehåller en katt eller hund.

Bildsegmentering: Att dela upp en bild i olika regioner eller segment, till exempel att segmentera en medicinsk bild för att identifiera olika organ eller vävnader.

Utmaningar inom Bildanalys

Bildanalys är en kraftfull teknik med många tillämpningar, men det finns flera utmaningar som forskare och ingenjörer måste övervinna för att uppnå framgångsrika resultat. Här är några av de främsta utmaningarna:

Storlek på Data:



Bilder innehåller enorma mängder data jämfört med andra typer av input (som text eller numerisk data). En högupplöst bild kan ha miljoner pixlar, vilket innebär att vi måste bearbeta och analysera stora datamängder.

Problem: Att bearbeta stora mängder bilddata kräver hög lagringskapacitet och beräkningskraft.

Exempel: Satellitbilder eller medicinska bildskanningar genererar stora datamängder, vilket gör det svårt att snabbt analysera dessa data utan avancerad hårdvara och optimerade algoritmer.

Komplexitet i Funktioner:



Bilder innehåller en mängd detaljer och mönster. Att extrahera användbara funktioner eller egenskaper från bilder för att utföra uppgifter som klassificering eller segmentering är mycket mer komplicerat än med strukturerad data som siffror eller text.

Problem: Enkla modeller kanske inte fångar upp alla relevanta detaljer, och avancerade metoder som neurala nätverk krävs för att förstå komplexa visuella mönster.

Exempel: Att känna igen ett ansikte under olika ljusförhållanden, vinklar, eller om personen har glasögon kan vara mycket utmanande.

Utmaningar inom Bildanalys

Variationsbredd i Bilder:



Samma objekt kan se väldigt olika ut beroende på olika faktorer som belysning, perspektiv, bakgrund och storlek. Detta kräver att modellerna är mycket robusta för att hantera olika typer av variationer.

Problem: Bildanalysmodeller måste kunna hantera brus, occlusion (när ett objekt är delvis blockerat) och andra oregelbundenheter i bilder.

Exempel: I handskrivna siffror (som i MNIST-datasetet) kan varje siffra ha olika skrivstilar, vilket gör klassificeringen svår. På samma sätt kan ansiktsigenkänning vara svårt med olika ansiktsuttryck eller ändrade vinklar.

Höga Krav på Beräkningskraft:



Bildanalysalgoritmer, särskilt djupa neurala nätverk som Convolutional Neural Networks (CNNs), kräver stor beräkningskraft för att träna och köra. Detta innebär att moderna GPUer (grafikkort) eller molntjänster ofta används för att bearbeta bilder.

Problem: Beräkningskraven kan vara en barriär för mindre företag eller forskare utan tillgång till dyra resurser. Dessutom kan det vara tidskrävande att träna modeller, särskilt på stora dataset.

Exempel: Träning av en djup CNN på en stor dataset som ImageNet kan ta dagar eller veckor, även med avancerad hårdvara som NVIDIA GPUer.

Neurala Nätverk och Bildanalys

Neurala nätverk, och särskilt djupa neurala nätverk (Deep Neural Networks, DNNs), är en familj av maskininlärningsmodeller inspirerade av den mänskliga hjärnans struktur. De består av flera lager av artificiella neuroner som lär sig komplexa mönster och samband från data. När det gäller bildanalys har dessa nätverk visat sig vara särskilt kraftfulla, eftersom de kan lära sig att automatiskt extrahera relevanta egenskaper och detaljer från bilder utan behov av manuell funktionsextraktion.

Neurala Nätverk i Bildanalys: Genom att stapla lager, särskilt Convolutional Neural Networks (CNNs), kan neurala nätverk automatiskt identifiera låg- och högnivådetaljer i bilder, som kanter, texturer, former och till slut mer abstrakta begrepp som objekt eller ansikten.



Varför är neurala nätverk lämpliga för bildanalys?

Automatisk funktionsextraktion: Neurala nätverk lär sig automatiskt att extrahera visuella funktioner som kanter och texturer, och bygger dessa till högre koncept som objekt och scener.

Skalbarhet: Neurala nätverk kan hantera stora mängder data, vilket är viktigt för bildanalys där stora dataset ofta används.

Robusthet: De kan hantera variationer i bilder, såsom förändringar i ljus, perspektiv, och bakgrund, mycket bättre än traditionella metoder.

Översikt av Konvolutionella Neurala Nätverk (CNN)

Konvolutionella neurala nätverk (CNN) är en typ av djupa neurala nätverk som är specialiserade på att analysera bilddata.

De är uppbyggda med tre nyckelmekanismer: **konvolution**, **pooling**, och **fullständiga (fully connected) lager**, vilket gör dem mycket kraftfulla på att identifiera och tolka visuella mönster. CNNs är idag standardverktyg inom allt från ansiktsigenkänning och självkörande bilar till medicinsk bilddiagnostik.

Convolution

- Ett litet filter (kernel) glider över bilden.
- Detta genererar en ny representation (feature map) av bilden genom att filtrera fram viktiga mönster som kanter, linjer, färger eller texturer.
- Konvolution hjälper CNN att fokusera på lokala visuella detaljer.

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114	328	-26	470	158
53	266	-61	-30	344
403	116	-47	295	244
108	-135	256	-128	344
314	346	279		

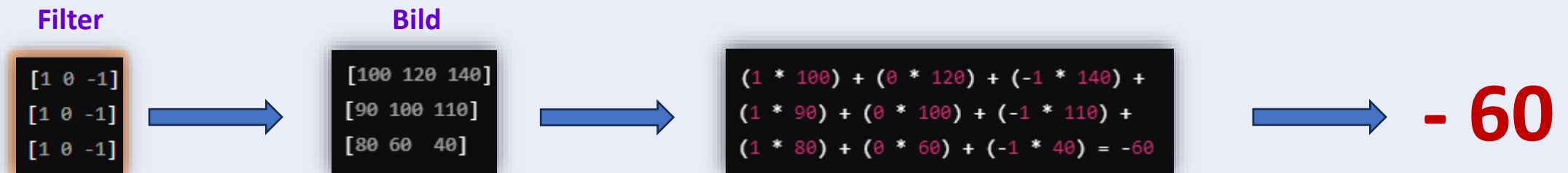
Convolution (CNN)

När vi pratar om konvolution i bildanalys och CNNs sker följande steg vid varje position:

Bildens pixelvärden: Varje bild består av ett rutnät av pixelvärden. Om det är en gråskalig bild är varje pixel representerad av ett numeriskt värde mellan 0 och 255 (där 0 är svart och 255 är vitt). För färgbilder finns det flera kanaler (Röd, Grön, Blå) där varje kanal har sitt eget rutnät av pixelvärden.

Filter (eller kernel): Ett filter, även kallat en kernel, är en mindre matris av tal, t.ex. 3x3 eller 5x5, som används för att extrahera specifika mönster från bilden, som kanter eller texturer.

Multiplikation vid varje position: Filtrets storlek bestämmer hur många pixlar som behandlas samtidigt. Om vi har ett 3x3 filter, tas 3x3 pixelvärden från bilden. Varje värde i detta 3x3 bildområde multipliceras med motsvarande värde i filtret.



Summan lagras: Denna summa (i vårt exempel -60) lagras som ett nytt pixelvärde i en ny matris (som representerar den bearbetade bilden). Detta processas för alla pixelpositioner i bilden, där filtret "glider" över bilden (en så kallad stridning).

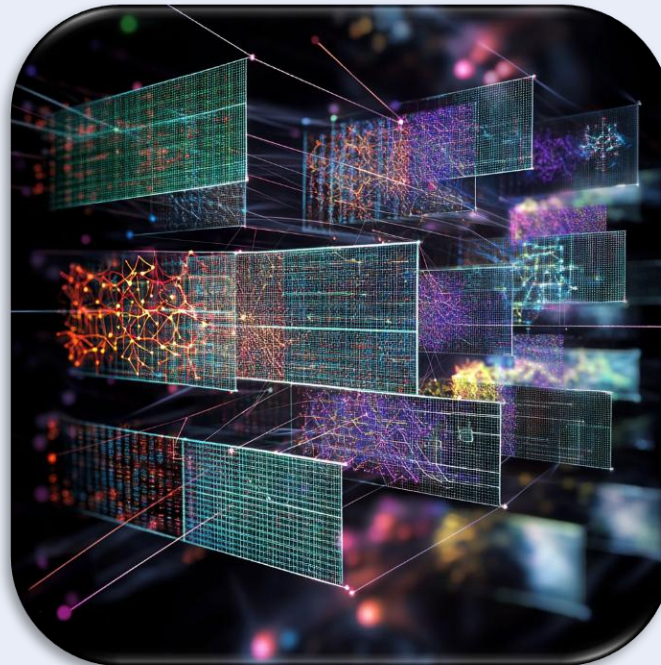
Konvolutionen hjälper datorn att upptäcka mönster som kanter och konturer, och detta görs genom att förstärka vissa funktioner i bilden och undertrycka andra beroende på hur filtret är uppbyggt. Konvolutionen är inte exakt samma som PCA eller UMAP, men båda förenklar datan och lyfter fram de viktigaste egenskaperna, fast på olika sätt och med olika mål.

Feature maps

När filtret (kernel) glider över bilden under **konvolutionsoperationen** genereras nya representationer av bilden kallade **feature maps**. Dessa är i praktiken nya bilder, där varje pixel indikerar hur starkt filtret reagerade på en specifik plats i originalbilden.

Om filtret exempelvis letar efter kanter, kommer feature map tydligt visa var dessa kanter finns i bilden.

Flera feature maps: CNN använder flera olika filter för att hitta olika egenskaper i bilden. Varje filter skapar sin egen feature map. Tillsammans ger dessa feature maps en samlad representation av olika egenskaper i bilden, som hjälper nätverket att förstå och tolka bilden.



Pooling

Pooling är ett steg i konvolutionella neurala nätverk (CNN) som används direkt efter konvolutionsstegen. Pooling minskar storleken på feature maps (de "kartor" med viktiga egenskaper i bilden), vilket gör det enklare, snabbare och mer effektivt att analysera bilddata.

Varför använder vi pooling?

1. Reducerar dimensionerna (downsampling):

- Gör datamängden mindre genom att endast behålla de viktigaste egenskaperna från varje del av bilden.
- Minskar därmed både beräkningskostnaden och risken för överanpassning (overfitting).

2. Gör modellen robustare:

- Nätverket blir mindre känsligt för små förändringar i bilden som position, rotation eller brus.
- Även om detaljer flyttas något kan nätverket ändå identifiera viktiga mönster tydligt.

Pooling

Max-pooling (vanligast förekommande):

- Bilden delas upp i små rutor (exempelvis 2x2 pixlar).
- Från varje ruta behåller nätverket endast det starkaste värdet (maxvärdet).
- Exempel: Om ett område innehåller en kant eller en annan tydlig struktur, kommer max-pooling att behålla denna struktur och ignorera mindre viktiga delar.

Exempel:

Tänk dig en bild av en katt.

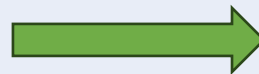
När vi använder max-pooling kan även små förskjutningar av katten i bilden ignoreras eftersom pooling fokuserar på de starkaste kännetecknen (t.ex. kanter, öron, ögon), vilket gör nätverket mer stabilt och precist även vid varierande input.

I sammanfattning hjälper pooling oss att:

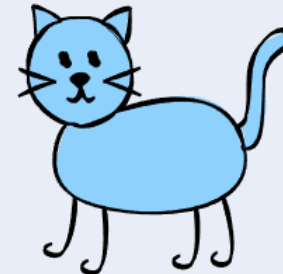
- Minska mängden data som måste bearbetas.
- Förbättra nätverkets effektivitet och minska risken för överanpassning.
- Göra nätverket mer motståndskraftigt mot små förändringar i bilden.



convolution



max pooling



Fullständiga Lager (Fully Connected Layers)

Fullständiga lager, eller Fully Connected Layers (FC-lager), är det sista steget i ett konvolutionellt neuralt nätverk (CNN) och liknar hur ett traditionellt neuralt nätverk fungerar.

Efter konvolution och pooling: När bilden har passerat genom flera konvolutionella lager och pooling-lager, har nätverket "lärt sig" många viktiga detaljer (eller funktioner) från bilden. Funktionen från varje lager samlas ihop till en lång lista med siffror. Dessa siffror representerar viktiga egenskaper hos bilden, som nätverket har lärt sig känna igen.

Ett fullständigt lager tar alla dessa siffror och bearbetar dem för att fatta ett beslut. Det betyder att varje neuron i det fullständiga lagret är kopplat till alla funktioner från det föregående lagret. I detta steg använder nätverket all information det lärt sig om bilden och försöker avgöra vilken kategori eller klass bilden tillhör.

Hur det fungerar i praktiken:

Tänk dig att vi har tränat ett nätverk för att känna igen olika typer av djur i bilder, som katter och hundar.

De tidigare lagren (konvolution och pooling) har lärt sig känna igen olika detaljer, som öron, tassar eller pälsmönster.

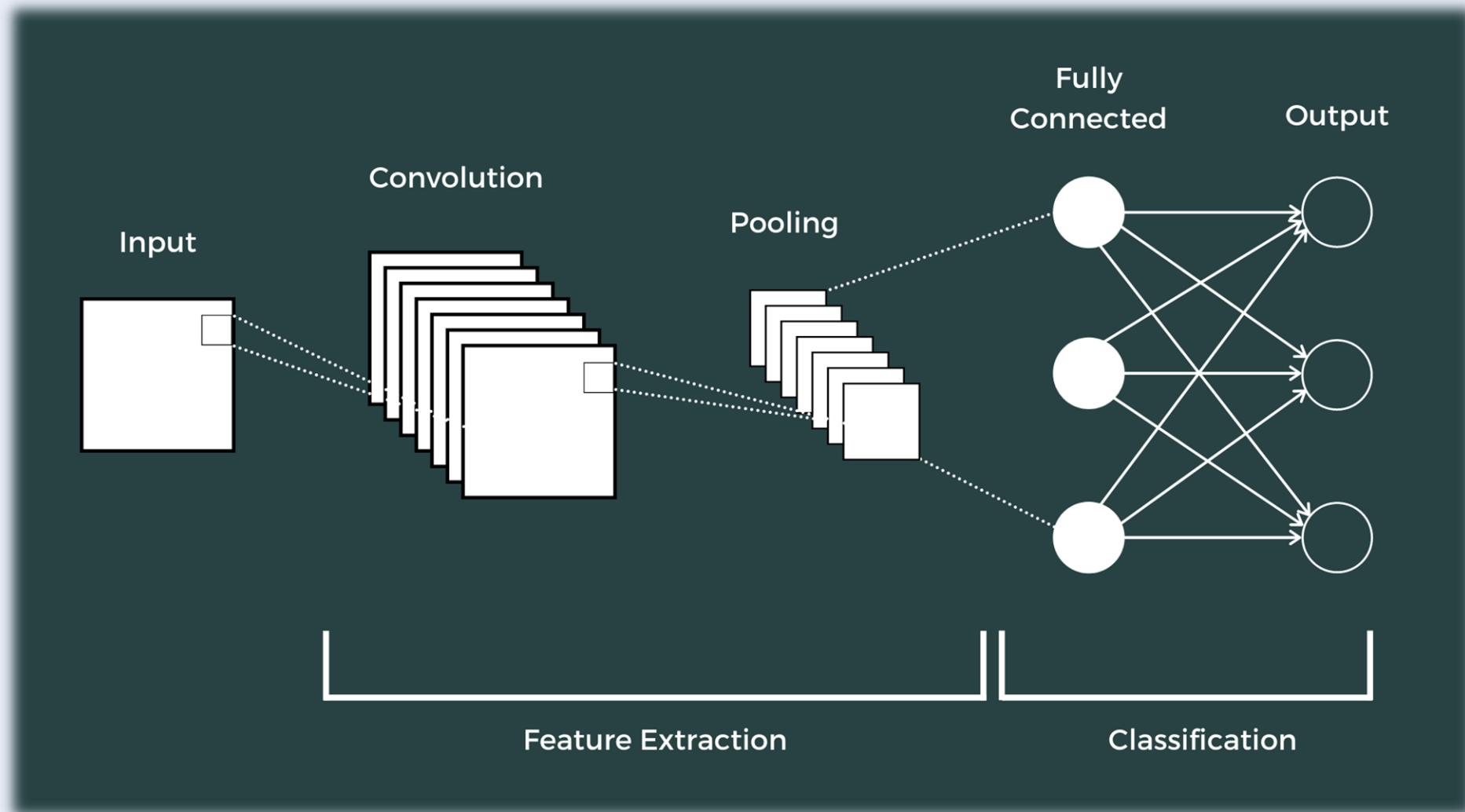
När den informationen når det fullständiga lagret, används den för att slutligen besluta: "Är det här en katt eller en hund?"

Slutlig klassificering: Efter att bilden har bearbetats i flera lager, ansvarar det fullständiga lagret för att ge en slutlig klassificering. Det avgör till exempel om en bild föreställer en katt, en hund eller något annat, beroende på vad nätverket har tränats att känna igen.

Sammanfattning:

Fullständiga lager är som en "beslutsdel" av nätverket. De tar alla viktiga funktioner som nätverket har extraherat från bilden och gör en slutlig bedömning om vad bilden innehåller.

CNN



Fördelar med CNN för Bildanalys

Automatiserad Funktionsextraktion:

CNNs behöver inte att manuellt designa och extrahera egenskaper från bilderna, vilket gör dem överlägsna traditionella metoder.

De lär sig automatiskt från data genom att identifiera mönster, former, och detaljer i bilden. Detta innebär att CNNs kan hitta både enkla och komplexa egenskaper (som kanter och former) genom flera lager av nätverket.

Translationsinvarians:

Translationsinvarians innebär att CNNs kan känna igen ett objekt oavsett var det finns i bilden. Det spelar ingen roll om ett objekt, som ett ansikte, är centrerat eller placerat i ett hörn – CNN kan fortfarande känna igen det.

Detta sker tack vare pooling-operationer och delade vikter, som gör nätverket robust mot förskjutningar och små förändringar i bildens position.

Effektiv Beräkning:

CNNs är mycket mer beräkningsmässigt effektiva jämfört med vanliga neurala nätverk för bildanalys.

Genom att använda konvolutioner med delade vikter, där samma filter används för olika delar av bilden, minskas antalet parametrar drastiskt. Detta sparar både minne och beräkningstid, samtidigt som nätverket kan tränas på större datamängder och djupare nätverk.

Nyckelbegrepp

Lokala Mottagningsfält:

Varje neuron i ett konvolutionellt lager bearbetar bara en liten del av bilden (lokalt område), vilket hjälper nätverket att fokusera på små, men viktiga detaljer.

Delade Vikter:

Ett filter (en uppsättning vikter) används över hela bilden, vilket innebär att samma mönster kan upptäckas på olika platser i bilden. Detta minskar antalet vikter som behöver tränas, vilket gör CNN effektivare och bättre på generalisering.

Bildförberedelse för Neurala Nätverk

Bildförberedelse inkluderar steg som normalisering, skalning, och omvandling av bilder till ett format som nätverket kan hantera.

Normalisering:

Bilderna behöver normaliseras för att göra att pixelvärdena hamnar inom ett jämnare och hanterbart intervall.

Vanligtvis består en bild av pixelvärden mellan 0 och 255. Genom att normalisera dessa värden till intervallet 0 till 1 (dela varje pixelvärde med 255), hjälper vi nätverket att konvergera snabbare under träning och undvika problem med stora värden som kan påverka viktuppdateringen.

Normalisering kan också hjälpa till att undvika problem med gradienter som exploderar eller försvinner.

```
X_train = X_train / 255.0  
X_test = X_test / 255.0
```

Bildförberedelse för Neurala Nätverk

Skalning:

Bildskalning innebär att alla bilder justeras till en specifik dimension som nätverket förväntar sig.

Neurala nätverk behöver att ***alla bilder har samma storlek***, så det är nödvändigt att skala eller omforma bilderna till en enhetlig storlek. Vanligtvis används dimensioner som 28x28 (för MNIST) eller 224x224 (för mer avancerade nätverk som ResNet eller VGG).

Exempel:

Om vi använder MNIST-data som är 28x28 pixlar i storlek, säkerställer vi att varje bild är korrekt dimensionerad före träning:

```
from tensorflow.keras.preprocessing.image import img_to_array, load_img
img = load_img('image.png', target_size=(28, 28))
img_array = img_to_array(img)
```

Bildförberedelse för Neurala Nätverk

Omformning till Tensorer:

- Bilder måste omvandlas till **tensorer**, vilket är det format som neurala nätverk kan arbeta med. **Tensorer** är flerdimensionella arrayer som representerar data.
- För en gråskalebild som MNIST kommer en bild omvandlas till en tensor av formen (28, 28, 1), där 1 representerar antalet färgkanaler (gråskala har bara en kanal). Färgade bilder (RGB) har tre kanaler och omvandlas till en tensor av formen (höjd, bredd, 3).

Exempel:

- Omvandla en 28x28-bild till en tensor:

```
X_train = X_train.reshape(-1, 28, 28, 1) # för gråskalebilder
```

- **värdet -1** har en speciell betydelse inom omformning av arrays eller tensorer i bibliotek som NumPy och TensorFlow. Det betyder att vi inte explicit anger hur många bilder som ska finnas i den första dimensionen, utan låter biblioteket automatiskt räkna ut detta baserat på den totala storleken på arrayen. Om vi har 60 000 träningsbilder, kommer -1 att motsvara 60 000.

Bildförberedelse för Neurala Nätverk

Dataaugmentation:

- **Dataaugmentation** används för att artificiellt öka storleken på träningsdatasetet genom att skapa modifierade versioner av bilderna (*genom rotation, förskjutning, spegling, etc.*). Detta hjälper till att göra modellen robustare mot små variationer i bilderna och minskar risken för överanpassning.
- Detta är ett viktigt steg i bildförberedelse om vi har ett litet dataset och vill förbättra modellens generaliseringsförmåga.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator  
datagen = ImageDataGenerator(rotation_range=10, width_shift_range=0.1,  
                             height_shift_range=0.1, zoom_range=0.1)
```

rotation_range=10: Detta anger att bilderna kan roteras med upp till ± 10 grader. Det betyder att varje bild som matas in i modellen under träningen kan roteras slumpmässigt inom detta intervall, vilket gör modellen mindre känslig för rotationsvariationer i de verkliga bilderna.

width_shift_range=0.1: Detta betyder att bilderna kan flyttas horisontellt (breddmässigt) med upp till 10% av bildens bredd. Om bilden exempelvis har en bredd på 28 pixlar (som i MNIST-datasetet), kan bilden flyttas upp till 2,8 pixlar åt höger eller vänster.

height_shift_range=0.1: Detta är samma som **width_shift_range**, men tillåter att bilden flyttas vertikalt (höjdmässigt) med upp till 10% av bildens höjd.

zoom_range=0.1: Detta innebär att bilderna kan zoomas in eller ut med upp till 10%. Modellen tränas med inzoomade eller utzoomade versioner av samma bild, vilket gör att den blir bättre på att hantera bilder där objektet kan vara i olika storlekar.

MNIST Dataset

MNIST Dataset

MNIST (Modified National Institute of Standards and Technology) är ett av de mest kända dataseten för att träna bildigenkänningssystem.

- Det innehåller 70 000 bilder av handskrivna siffror (0–9).
- Datasetet består av 60 000 träningsbilder och 10 000 testbilder, där varje bild är 28x28 pixlar i storlek och i gråskala.

Användning inom Djupinlärning:

- MNIST används ofta som ett "hello world"-exempel inom bildklassificering och djupinlärning.
- Eftersom det är relativt enkelt och litet, är det perfekt för att lära sig grunderna i bildigenkänning och för att testa olika typer av neurala nätverk, speciellt CNNs.

Varför är MNIST Viktigt?

- Datasetet är enkelt men ändå tillräckligt komplext för att ge insikter om hur djupa neurala nätverk kan byggas och tränas.
- Det är en standard för benchmarking av algoritmer och modeller inom bildklassificering.
- Tack vare sin lilla storlek kan det snabbt användas för att experimentera med olika modeller utan behov av stor beräkningskraft.

Struktur och Egenskaper:

- Varje bild i datasetet är en siffra (0 till 9) i olika handstilar.
- Bilderna är normaliserade och centrerade, vilket betyder att varje siffra är placerad ungefär i mitten av bilden.
- Bilderna är i gråskala, vilket innebär att varje pixel har ett värde mellan 0 och 255, där 0 är svart och 255 är vitt.

Exempel på Användning:

- Klassificera handskrivna siffror för automatisk läsning av postnummer, bankcheckar och liknande.
- Lämpligt för att testa olika djupinlärningsarkitekturer, som konvolutionella nätverk (CNNs).

MNIST Dataset

Handskrivna siffror (0-9)



Ladda MNIST Dataset

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
```

```
# Ladda datasetet
(X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()

# Normalisera bilderna till ett intervall mellan 0 och 1
X_train, X_test = X_train / 255.0, X_test / 255.0
```

MNIST-datasetet finns direkt tillgängligt via Keras API och består av tränings- och testdata.

load_data(): Hämtar tränings- och testdata (bilder och deras etiketter).

Normalisering: Eftersom pixelvärdena är mellan 0 och 255, normaliserar vi dem till mellan 0 och 1 för att hjälpa nätverket att konvergera snabbare.

TF 2.19.0: WSL

Ladda MNIST Dataset

```
import os
os.environ['XLA_FLAGS'] = '--xla_gpu_cuda_data_dir=/usr/local/cuda'
# set an environment variable to help TensorFlow find CUDA
```

```
import tensorflow as tf
print("TensorFlow version:", tf.__version__)
print("Num GPUs Available: ", len(tf.config.experimental.list_physical_devices('GPU')))

from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
```

```
TensorFlow version: 2.19.0
Num GPUs Available: 1
```

```
# Limit GPU memory usage explicitly
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        print("GPU memory growth enabled.")
    except RuntimeError as e:
        print(e)
```

Om bilderna är i mappar på dator, vad händer då?

```
data/  
  0/  
    img1.png  
    img2.png  
    ...  
  1/  
    img1.png  
    img2.png  
    ...  
  2/  
    img1.png  
    img2.png  
    ...
```

Varje underkatalog (0, 1, 2, ...) representerar en siffra och innehåller bilder som föreställer just den siffran. Bilderna måste ha ett korrekt format (som PNG eller JPEG).

Ladda bilder från mappar och etiketter:

För att ladda in bilderna och deras etiketter kan vi använda Keras inbyggda funktion ***flow_from_directory***.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator  
  
# Definiera datageneratoren och normalisera bilderna (skalning mellan 0 och 1)  
datagen = ImageDataGenerator(rescale=1.0/255.0)  
  
# Ladda träningsbilder från katalogen 'data/' med batchstorlek 32 och gråskala  
train_data = datagen.flow_from_directory(  
    'data/',          # Mapp där data finns  
    target_size=(28, 28), # För att matcha bildstorleken  
    color_mode='grayscale', # Om det är gråskalebilder  
    batch_size=32,      # Hur många bilder som laddas in per batch  
    class_mode='categorical' # Eftersom vi har fler än två klasser  
)
```

'data/': Den överordnade mappen där alla underkataloger (0, 1, 2, ...) finns.
target_size=(28, 28): För att skala om bilderna till 28x28 pixlar, precis som i MNIST.

color_mode='grayscale': Eftersom vi arbetar med gråskalebilder, kommer vi att omvandla dem till gråskala.

batch_size=32: Hur många bilder som ska laddas in åt gången. Det hjälper att använda batchar istället för att ladda in hela datasetet på en gång, eftersom det kan bli minneskrävande.

class_mode='categorical': Eftersom vi har fler än två klasser, konverterar vi etiketterna till "one-hot encoding" (exempelvis [0, 1, 0] för klassen 1).

Om bilderna är i mappar på dator, vad händer då?

Med hjälp av `train_data` får vi inte bara bilderna utan också deras etiketter automatiskt.

```
# Hämta första batchen av bilder och deras etiketter
X_batch, y_batch = next(train_data)

# Kontrollera formerna
print(X_batch.shape) # Ex: (32, 28, 28, 1) - batch_size, höjd, bredd, kanaler
print(y_batch.shape) # Ex: (32, 10) - batch_size, antal klasser (one-hot encoded)
```

X_batch: Innehåller bilderna i batchen, som är skalade och omformade.

y_batch: Innehåller motsvarande one-hot encoded etiketter (om du har 10 klasser, blir etiketten för siffran 2 [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]).

next()-funktionen används här eftersom `train_data` är en generator som kontinuerligt producerar batchar av data när vi behöver dem.

```
root_directory/
├── train/
│   ├── class1/          # Mappar för varje klass
│   │   ├── image1.jpg
│   │   ├── image2.jpg
│   │   └── ...
│   ├── class2/
│   │   ├── image1.jpg
│   │   ├── image2.jpg
│   │   └── ...
│   └── ...
└── test/
    ├── class1/
    │   ├── image1.jpg
    │   ├── image2.jpg
    │   └── ...
    ├── class2/
    │   ├── image1.jpg
    │   ├── image2.jpg
    │   └── ...
    └── ...
```

Omformning av data för CNN

Convolutionella Neurala Nätverk (CNNs) förväntar sig data i form av 4D-tensorer, dvs (antal, höjd, bredd, kanaler).

```
# Lägg till en kanal (gråskala)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
```

reshape(-1, 28, 28, 1): Vi lägger till en kanal för gråskala-bilder. Formatet är (antal bilder, höjd, bredd, kanaler).

CNN-modell

Nu skapar vi en CNN med några konvolutionella lager och pooling-lager, följt av fullt anslutna lager (fully connected layers).

```
model = models.Sequential()

# Första konvolutionella lagret
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2)))

# Andra konvolutionella lagret
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Tredje konvolutionella lagret
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

Conv2D: Konvolutionella lager som extraherar features från bilderna. Här används ett 3x3 filter och 32 respektive 64 filter.

MaxPooling2D: Pooling-lager för att minska dimensionerna på feature maps och kontrollera överanpassning.

CNN modell: filter

När vi säger att **Conv2D(32, (3, 3))** används, betyder det att 32 olika filter (eller kärnor) används. Varje filter är en unik 3x3 matris med olika vikter som lärts under träningen. Filtrena är utformade för att upptäcka olika egenskaper i bilden.

Viktiga koncept:

Unika filter: Varje av dessa 32 filter har olika vikter och är därför tränade för att upptäcka olika mönster. Vissa filter kan vara känsliga för horisontella kanter, andra för vertikala kanter, och ytterligare andra för mer komplexa egenskaper som texturer, former, eller färgövergångar.

Feature maps: Varje filter kommer att skapa en egen feature map (funktionskarta), som visar var i bilden just det specifika filtret har hittat de mönster det letar efter. Dessa feature maps representerar olika aspekter av bilden, baserat på vad varje filter har "upptäckt".

Olika resultat från olika filter: Eftersom varje filter är unikt och tränas för att upptäcka olika typer av mönster, kommer varje feature map att vara olika. Det innebär att en feature map kan visa var det finns horisontella kanter, medan en annan kan visa var det finns texturer eller skuggor i bilden.

Filtrens vikter initieras slumpmässigt och justeras sedan gradvis genom backpropagation och gradient descent under träningen.

För varje träningsbild beräknar nätverket sin förutsägelse, mäter hur fel förutsägelsen är med hjälp av en förlustfunktion, och justerar vikterna i filtren för att minimera felet i framtida förutsägelser.

Genom att uppdatera vikterna iterativt lär sig nätverket att upptäcka specifika egenskaper i bilderna och gör bättre förutsägelser.

Pooling-lager: MaxPooling2D

```
model.add(layers.MaxPooling2D((2, 2)))
```

MaxPooling är en typ av pooling-operation där det största värdet (max-värdet) från en liten region i en feature map tas ut. Pooling är ett sätt att reducera dimensionerna på feature maps samtidigt som viktiga informationsegenskaper behålls.

2D indikerar att detta är en pooling-operation som används på tvådimensionella data, som bilder.

(2, 2) anger storleken på "fönstret" eller regionen där pooling-operationen appliceras. I detta fall innebär det att nätverket tar ett område på 2x2 pixlar i feature mapen och väljer det största värdet inom detta område.

Fönstret förflyttas över hela feature mapen, och varje 2x2-område reduceras till en enskild pixel med det högsta värdet från det området.

CNN-modell

```
# Flattening av lagret för fullt anslutet nätverk  
model.add(layers.Flatten())
```

Flatten: Omformar data till en vektor för att kunna skickas in i fullt anslutna lager.

Flattening innebär att man tar alla element från de två- eller tredimensionella feature maps och lägger dem i en lång endimensionell lista (vektor).

När används Flatten?

Efter flera konvolutionella och pooling-lager har feature maps fortfarande en två- eller tredimensionell struktur, som till exempel kan vara 7x7x64. För att kunna koppla detta till ett fullt anslutet lager, som förväntar sig en endimensionell indata, behöver vi "platta ut" denna struktur.

Exempel:

Anta att vi har en **7x7x64-feature map** efter konvolutionella och pooling-lager:

7: Höjd på feature map.

7: Bredd på feature map.

64: Antal filter (djupet på feature maps).

Efter att vi applicerar Flatten(), omvandlas denna till en endimensionell vektor:

Från **7x7x64** -> **7 * 7 * 64 = 3,136 element**.

Dessa 3,136 värden används sedan som indata till det första fullt anslutna lagret i nätverket.

```
[0.0, 1.2, -0.5, 0.3, 0.9, 0.0, 2.1, ..., -0.8, 1.5, 0.0]
```

Varje element (siffra) i denna vektor är ett resultat av filtren som har bearbetat olika delar av bilden. Denna lista är alltså en abstrakt representation av hela bilden, där mönster och detaljer från bilden har "komprimerats" och "extraherats". Det är denna vektor som vi sedan använder för att fatta beslut om vad bilden föreställer.

CNN-modell

```
# Första fullt anslutna lagret  
model.add(layers.Dense(64, activation='relu'))  
  
# Utgångslagret  
model.add(layers.Dense(10, activation='softmax'))
```

Dense: Ett fullt anslutet lager där varje neuron är ansluten till alla neuroner i föregående lager.

Det sista lagret har 10 neuroner för de 10 klasserna (siffrorna 0–9).

TF 2.19.0: WSL

CNN-modell

```
model = models.Sequential([
    layers.Input(shape=(28, 28, 1)), # Rekommenderad syntax för input-lagret
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Kompilera Modellen

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

Adam: En populär optimeringsalgoritm som justerar inlärningshastigheten under träning.

sparse_categorical_crossentropy: Förlustfunktionen som används när vi har kategoriska klasser (som siffror 0–9).

accuracy: Mätvärdet som nätverket utvärderar under träning.

Träna Modellen

```
history = model.fit(X_train, y_train, epochs=10,  
                    validation_data=(X_test, y_test))
```

```
Epoch 1/10  
1875/1875 [=====] - 25s 3ms/step - loss: 0.1470 - accuracy: 0.9541 - val_loss: 0.0536 - val_accuracy: 0.9841  
Epoch 2/10  
1875/1875 [=====] - 5s 2ms/step - loss: 0.0486 - accuracy: 0.9848 - val_loss: 0.0333 - val_accuracy: 0.9892  
Epoch 3/10  
1875/1875 [=====] - 5s 2ms/step - loss: 0.0336 - accuracy: 0.9893 - val_loss: 0.0618 - val_accuracy: 0.9817  
Epoch 4/10  
1875/1875 [=====] - 5s 2ms/step - loss: 0.0269 - accuracy: 0.9913 - val_loss: 0.0310 - val_accuracy: 0.9903  
Epoch 5/10  
1875/1875 [=====] - 5s 2ms/step - loss: 0.0195 - accuracy: 0.9939 - val_loss: 0.0288 - val_accuracy: 0.9909  
Epoch 6/10  
1875/1875 [=====] - 5s 2ms/step - loss: 0.0161 - accuracy: 0.9952 - val_loss: 0.0265 - val_accuracy: 0.9923  
Epoch 7/10  
1875/1875 [=====] - 5s 2ms/step - loss: 0.0138 - accuracy: 0.9956 - val_loss: 0.0255 - val_accuracy: 0.9921  
Epoch 8/10  
1875/1875 [=====] - 5s 2ms/step - loss: 0.0111 - accuracy: 0.9962 - val_loss: 0.0334 - val_accuracy: 0.9907  
Epoch 9/10  
1875/1875 [=====] - 5s 3ms/step - loss: 0.0093 - accuracy: 0.9974 - val_loss: 0.0302 - val_accuracy: 0.9919  
Epoch 10/10  
1875/1875 [=====] - 5s 3ms/step - loss: 0.0099 - accuracy: 0.9967 - val_loss: 0.0353 - val_accuracy: 0.9907
```

epochs=10: Antalet gånger vi ska träna modellen på hela träningsdatasetet.

validation_data: Testdata används för att utvärdera modellen under träning, men den används inte för att justera vikterna.

model.fit(), returnerar den ett objekt av typen **History**. Detta objekt innehåller:

- Träningsförlust och metrik (t.ex. noggrannhet) för varje epoch.
- Valideringsförlust och metrik (om valideringsdata är angiven) för varje epoch.

Träna Modellen

```
Epoch 1/10
1875/1875 [=====] - 25s 3ms/step - loss: 0.1470 - accuracy: 0.9541 - val_loss: 0.0536 - val_accuracy: 0.9841
Epoch 2/10
1875/1875 [=====] - 5s 2ms/step - loss: 0.0486 - accuracy: 0.9848 - val_loss: 0.0333 - val_accuracy: 0.9892
Epoch 3/10
1875/1875 [=====] - 5s 2ms/step - loss: 0.0336 - accuracy: 0.9893 - val_loss: 0.0618 - val_accuracy: 0.9817
Epoch 4/10
1875/1875 [=====] - 5s 2ms/step - loss: 0.0269 - accuracy: 0.9913 - val_loss: 0.0310 - val_accuracy: 0.9903
Epoch 5/10
1875/1875 [=====] - 5s 2ms/step - loss: 0.0195 - accuracy: 0.9939 - val_loss: 0.0288 - val_accuracy: 0.9909
Epoch 6/10
1875/1875 [=====] - 5s 2ms/step - loss: 0.0161 - accuracy: 0.9952 - val_loss: 0.0265 - val_accuracy: 0.9923
Epoch 7/10
1875/1875 [=====] - 5s 2ms/step - loss: 0.0138 - accuracy: 0.9956 - val_loss: 0.0255 - val_accuracy: 0.9921
Epoch 8/10
1875/1875 [=====] - 5s 2ms/step - loss: 0.0111 - accuracy: 0.9962 - val_loss: 0.0334 - val_accuracy: 0.9907
Epoch 9/10
1875/1875 [=====] - 5s 3ms/step - loss: 0.0093 - accuracy: 0.9974 - val_loss: 0.0302 - val_accuracy: 0.9919
Epoch 10/10
1875/1875 [=====] - 5s 3ms/step - loss: 0.0099 - accuracy: 0.9967 - val_loss: 0.0353 - val_accuracy: 0.9907
```

Efter att varje epoch är klar, ger TensorFlow/Keras en uppdatering på modellens prestanda:

- **loss:** Förlusten (loss) är ett mått på hur dåligt modellen presterar på träningsdatan. Här är förlusten i första epoch 0.1470 och den minskar i senare epoch.
- **accuracy:** Detta är träningsnoggrannheten, som är andelen korrekt klassificerade exempel i träningsdatan. I första epochen är träningsnoggrannheten 0.9541 (dvs 95.41 %).
- **val_loss:** Förlusten på valideringsdatan (dvs testdatan), som används för att utvärdera modellens prestanda på data den inte har sett under träningen. I första epoch är valideringsförlusten 0.0536.
- **val_accuracy:** Valideringsnoggrannheten (dvs på testdatan), som i första epochen är 0.9841 (dvs 98.41 %). Detta visar hur väl modellen generaliserar till ny data.

TF 2.19.0: WSL

Träning med Early Stopping

```
from tensorflow.keras.callbacks import EarlyStopping

early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test),
    callbacks=[early_stopping]
)
```


Utvärdera Modellen

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test accuracy: {test_acc}")
```

```
313/313 [=====] - 1s 3ms/step - loss: 0.0353 - accuracy: 0.9907
Test accuracy: 0.9907000064849854
```

Den här funktionen används för att utvärdera modellen på en uppsättning testdata (eller valideringsdata) som modellen inte har sett under träningen.

Input-parametrar:

X_test: Testdata, dvs bilderna som används för att utvärdera hur väl modellen fungerar på osett data.

y_test: Etiketterna (labels) för testdata, dvs de faktiska klasserna som de här bilderna tillhör.

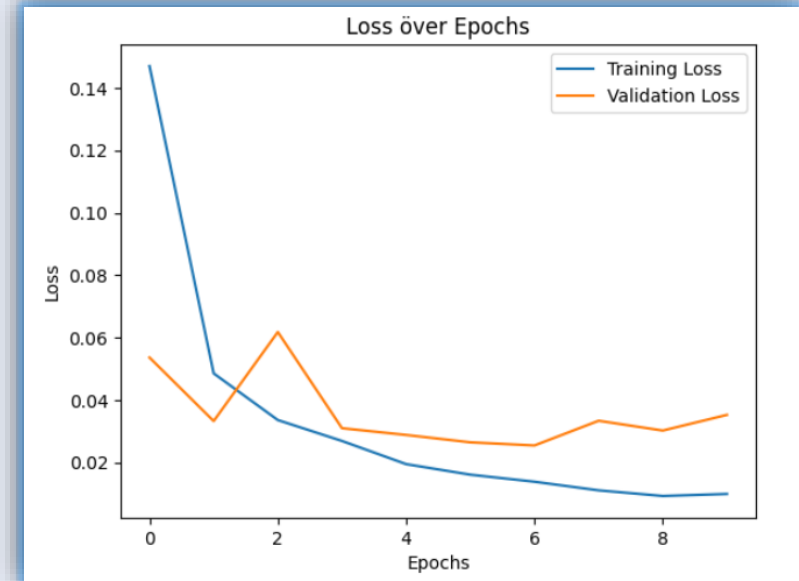
Returnerar:

test_loss: Förlusten på testdatan (hur felaktiga modellens förutsägelser är).

test_acc: Testnoggrannheten (andelen korrekt klassificerade bilder på testdatan).

Visualisera Träningshistoriken

```
# Tränings- och valideringsförlust
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss över Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



Initial förlust (Epok 0-1): Vid början har både tränings- och valideringsförlusten ett högre värde (över 0.14). Detta är normalt eftersom modellen precis har börjat lära sig och ännu inte har optimerat sina vikter.

Snabb minskning av förlust (Epok 0-3):

Förlusten på både tränings- och valideringsdatan minskar snabbt under de första få epokerna, vilket innebär att modellen snabbt förbättrar sin prestanda och lär sig mönster i data.

Stabilisering (Efter Epok 4):

Efter de första epokerna börjar träningsförlusten minska stabilt medan valideringsförlusten också minskar, men i ett långsammare tempo. Detta kan betyda att modellen har anpassat sig väl till träningen men fortfarande justeras för att prestera bättre på valideringsdatan.

Valideringsförlustens fluktuationer:

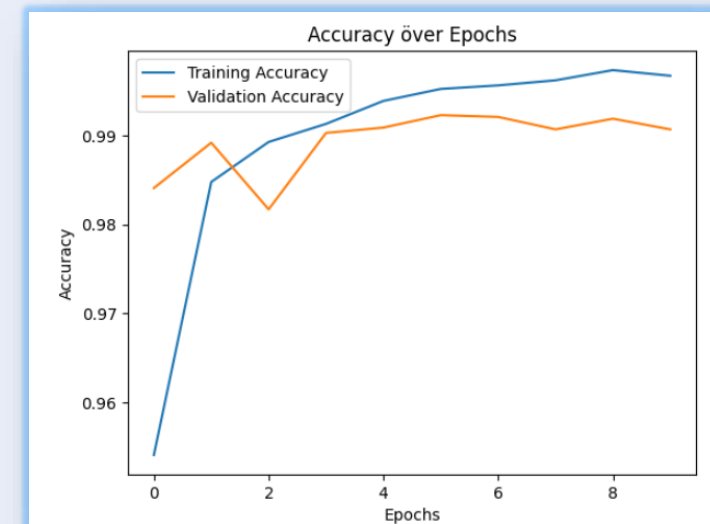
Valideringsförlusten verkar variera mer än träningsförlusten, särskilt efter de första epokerna. Detta är normalt och kan bero på att modellen ser olika exempel från valideringsdatan under varje epok.

Ingen tydlig överanpassning (overfitting):

Modellen verkar inte överanpassa (overfitting) i detta fall eftersom valideringsförlusten inte ökar dramatiskt eller divergerar kraftigt från träningsförlusten efter flera epoker. De följer varandra på ett stabilt sätt, vilket är ett tecken på att modellen generaliserar bra.

Visualisera Träningshistoriken

```
# Tränings- och valideringsprecision
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy över Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```



Snabb förbättring i början (Epok 0-2):

Under de första två epokerna förbättras både träningsnoggrannheten och valideringsnoggrannheten snabbt. Detta visar att modellen snabbt lär sig viktiga mönster i datan.

Stabilisering (Efter Epok 3):

Efter den tredje epoken börjar noggrannheten på både tränings- och valideringsdata att stabiliseras runt 99%. Det innebär att modellen uppnår en mycket hög noggrannhet, och ytterligare epoker leder till gradvisa små förbättringar snarare än stora förändringar.

Träningsnoggrannheten når nära 100%:

Träningsnoggrannheten når nästan 100% under de sista epokerna. Detta innebär att modellen lyckas korrekt förutsäga de allra flesta exemplen i träningsdatan.

Valideringsnoggrannheten ligger stabilt runt 99%:

Valideringsnoggrannheten stabiliserar sig något under träningsnoggrannheten, vilket är normalt. Det innebär att modellen presterar nästan lika bra på ny, osedd data som den gör på träningsdata.

Ingen överanpassning (overfitting):

Valideringsnoggrannheten ligger nära träningsnoggrannheten, vilket innebär att modellen inte överanpassar sig till träningsdata. Om valideringsnoggrannheten hade varit mycket lägre än träningsnoggrannheten skulle det ha varit ett tecken på överanpassning.

Modellens prestanda

- Modellen har lärt sig snabbt under de första epokerna, och både tränings- och valideringsförlusten minskar stadigt.
- Inga tydliga tecken på överanpassning, vilket betyder att modellen fungerar bra både på träningsdatan och på ny data (valideringsdatan).
- Vi kan överväga att testa längre träning för att se om ytterligare förbättringar kan göras, eller använda tekniker som **early stopping** för att säkerställa att träningen avslutas när modellen når sin bästa prestanda.
- Modellen har lärt sig mycket bra och uppnår en mycket hög noggrannhet både på träningsdatan och valideringsdatan (runt 99%).
- Detta är en väldigt bra prestanda, och vi kan överväga att stanna vid dessa epoker eftersom modellen redan presterar nära optimalt.

Förutsägelser på testdata

```
# Gör förutsägelser på testdatan
predictions = model.predict(X_test)

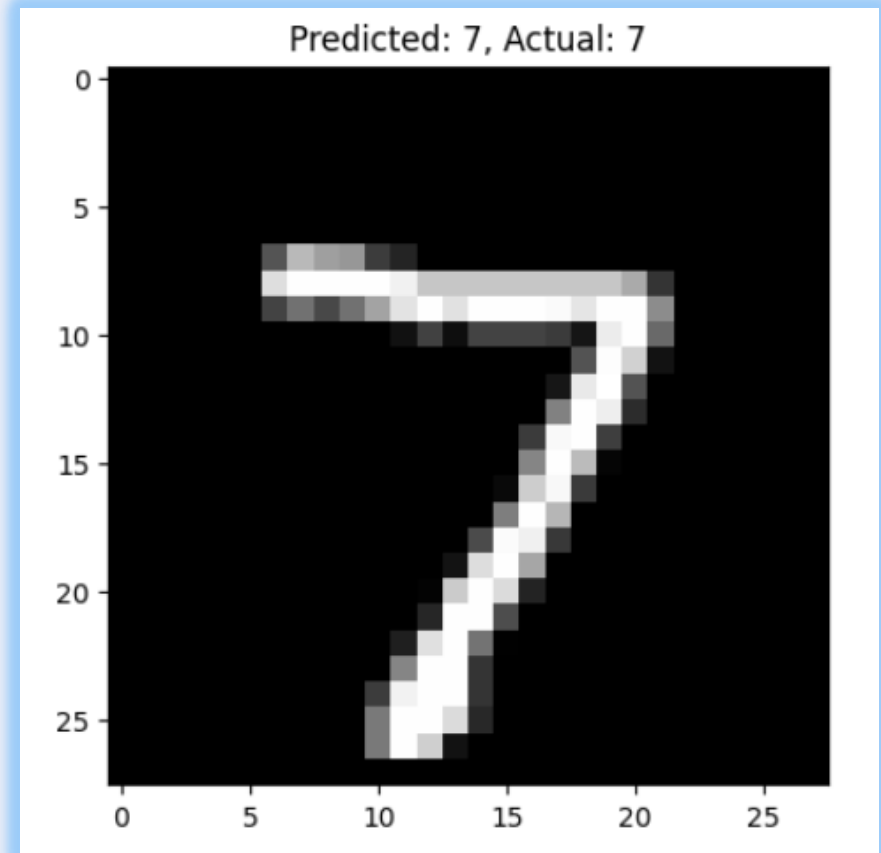
# Visa en exempelbild från testdatan och modellens förutsägelse
import numpy as np
plt.imshow(X_test[0].reshape(28, 28, 1), cmap='gray')
plt.title(f"Predicted: {np.argmax(predictions[0])}, Actual: {y_test[0]}")
plt.show()
```

Bildvisning: Denna del av koden visar den första bilden i testdatan (`X_test[0]`). Bilden återformas till en 28x28 bild i gråskalan med hjälp av `.reshape(28, 28, 1)`.

Prediktion och verkligt värde: Titeln på grafen visar modellens förutsägelse (Predicted) och det verkliga värdet (Actual) för bilden.

`np.argmax(predictions[0])`: Tar fram den klass (siffra) som har högst sannolikhet i modellens förutsägelse.

`y_test[0]`: Det verkliga siffervärdet för bilden i testdatan.



Modell utvärdering: Confusion matrix

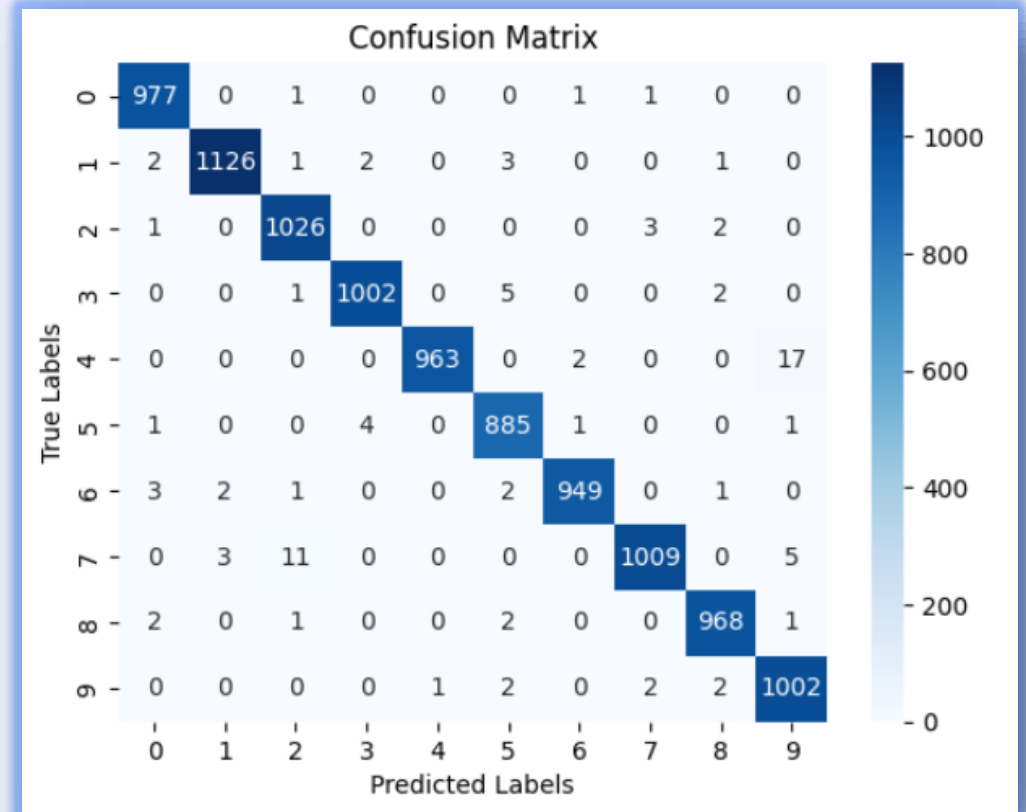
```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import numpy as np

# Gör förutsägelser på testdatan
predictions = model.predict(X_test)

# Omvandla förutsägelserna till klassetiketter (argmax för att få den högsta sannolikheten)
y_pred_classes = np.argmax(predictions, axis=1)

# Skapa confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred_classes)

# Visualisera confusion matrix med heatmap
sns.heatmap(conf_matrix, annot=True, cmap='Blues', fmt='g')
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.show()
```



Modell utvärdering: Confusion matrix

```
import numpy as np

# Confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred_classes)

# Antal klasser (för MNIST är det 10 klasser, 0-9)
num_classes = conf_matrix.shape[0]

# Initialisera tomma listor för TP, FP, FN och TN för varje klass
TP = []
FP = []
FN = []
TN = []

# Loopa igenom varje klass för att beräkna TP, FP, FN och TN
for i in range(num_classes):
    # True Positives (diagonal)
    tp = conf_matrix[i, i]
    # False Positives (kolumnen, exklusive diagonalen)
    fp = conf_matrix[:, i].sum() - tp
    # False Negatives (raden, exklusive diagonalen)
    fn = conf_matrix[i, :].sum() - tp
    # True Negatives (allting annat)
    tn = conf_matrix.sum() - (tp + fp + fn)

    # Lägg till värdena till listorna
    TP.append(tp)
    FP.append(fp)
    FN.append(fn)
    TN.append(tn)

# Skriv ut resultatet för varje klass
for i in range(num_classes):
    print(f"Class {i}:")
    print(f" TP: {TP[i]}, FP: {FP[i]}, FN: {FN[i]}, TN: {TN[i]}")
```

```
Class 0:
    TP: 977, FP: 9, FN: 3, TN: 9011
Class 1:
    TP: 1126, FP: 5, FN: 9, TN: 8860
Class 2:
    TP: 1026, FP: 16, FN: 6, TN: 8952
Class 3:
    TP: 1002, FP: 6, FN: 8, TN: 8984
Class 4:
    TP: 963, FP: 1, FN: 19, TN: 9017
Class 5:
    TP: 885, FP: 14, FN: 7, TN: 9094
Class 6:
    TP: 949, FP: 4, FN: 9, TN: 9038
Class 7:
    TP: 1009, FP: 6, FN: 19, TN: 8966
Class 8:
    TP: 968, FP: 8, FN: 6, TN: 9018
Class 9:
    TP: 1002, FP: 24, FN: 7, TN: 8967
```

Modell utvärdering: Classification report

```
from sklearn.metrics import classification_report

# Förutsägelser på testdatan
y_pred_classes = model.predict(X_test).argmax(axis=1)

# Skriv ut classification report
print(classification_report(y_test, y_pred_classes))
```

	precision	recall	f1-score	support
0	0.99	1.00	0.99	980
1	1.00	0.99	0.99	1135
2	0.98	0.99	0.99	1032
3	0.99	0.99	0.99	1010
4	1.00	0.98	0.99	982
5	0.98	0.99	0.99	892
6	1.00	0.99	0.99	958
7	0.99	0.98	0.99	1028
8	0.99	0.99	0.99	974
9	0.98	0.99	0.98	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

Modell utvärdering: ROC AUC

```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

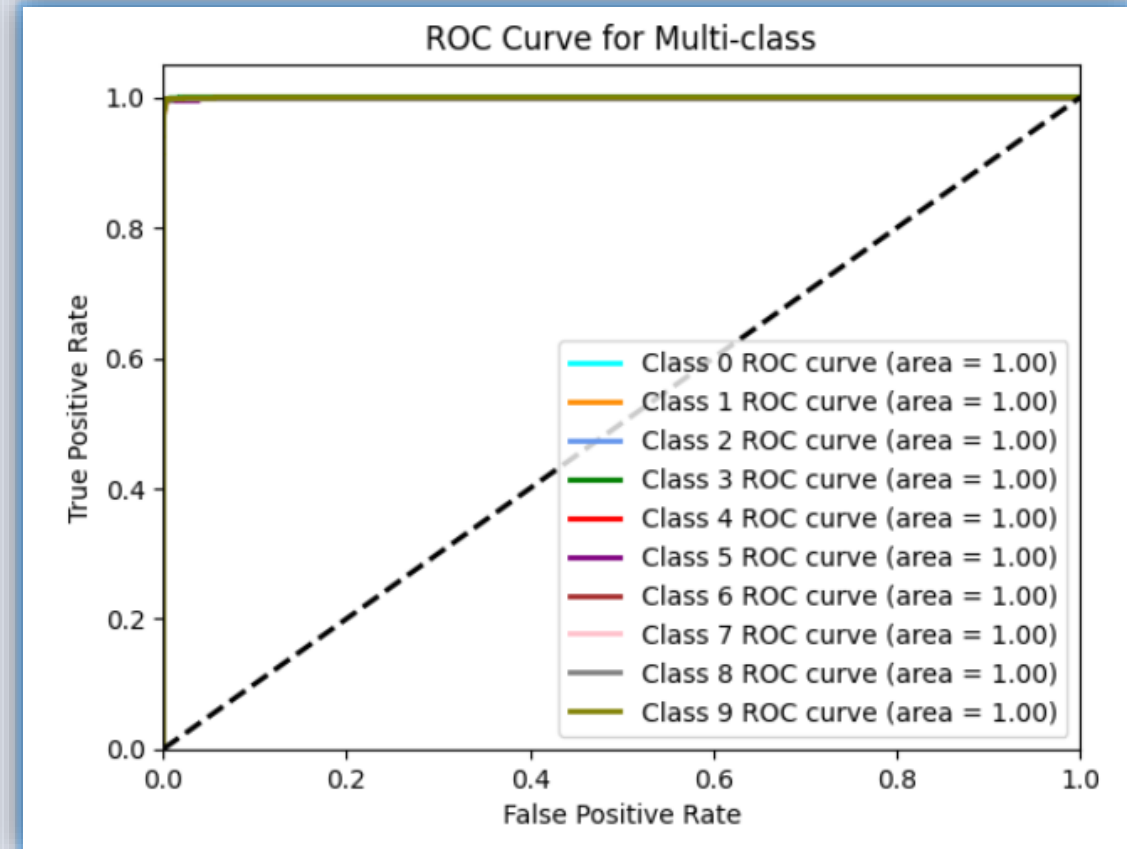
# Binarisera y_test för multiclass ROC
y_test_binarized = label_binarize(y_test, classes=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y_pred_prob = model.predict(X_test)

# ROC-kurva och AUC
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(10): # eftersom det är 10 klasser
    fpr[i], tpr[i], _ = roc_curve(y_test_binarized[:, i], y_pred_prob[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

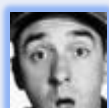
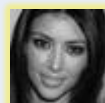
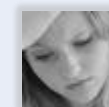
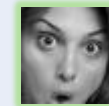
# Plot ROC kurvor för varje klass
plt.figure()
colors = ['aqua', 'darkorange', 'cornflowerblue', 'green', 'red', 'purple', 'brown', 'pink', 'gray', 'olive']
for i, color in zip(range(10), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2, label=f'Class {i} ROC curve (area = {roc_auc[i]:0.2f})')

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Multi-class')
plt.legend(loc="lower right")
plt.show()
```



Kaggle: FER-2013

Facial expressions



FER-2013

The data consists of 48x48 pixel grayscale images of faces. The faces have been automatically registered so that the face is more or less centred and occupies about the same amount of space in each image.

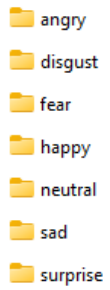
The task is to categorize each face based on the emotion shown in the facial expression into one of seven categories (**0=Angry, 1=Disgust, 2=Fear, 3=Happy, 4=Sad, 5=Surprise, 6=Neutral**).

The **training set** consists of 28,709 examples and the public test set consists of 3,589 examples.

FER-2012

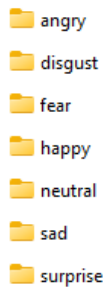
Dataset på hårddisken:

train



- angry
- disgust
- fear
- happy
- neutral
- sad
- surprise

test



- angry
- disgust
- fear
- happy
- neutral
- sad
- surprise

Klass	Antal
happy	7215
sad	4830
neutral	4965
angry	3995
surprise	3171
fear	4097
disgust	436

ImageDataGenerator för att ladda datasetet

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Definiera ett seed-värde för reproducerbarhet
seed_value = 42

# Generator för träning och validering med seed
train_val_datagen = ImageDataGenerator(
    rescale=1.0/255.0,
    validation_split=0.2
)

# Separat generator för test (utan split) med samma seed
test_datagen = ImageDataGenerator(rescale=1.0/255.0)

# Träningsgenerator
train_generator = train_val_datagen.flow_from_directory(
    './train',
    target_size=(48, 48),
    color_mode='grayscale',
    batch_size=32,
    class_mode='categorical',
    subset='training',
    seed=seed_value
)

# Valideringsgenerator
validation_generator = train_val_datagen.flow_from_directory(
    './train',
    target_size=(48, 48),
    color_mode='grayscale',
    batch_size=32,
    class_mode='categorical',
    subset='validation',
    seed=seed_value
)

# Testgenerator
test_generator = test_datagen.flow_from_directory(
    './test',
    target_size=(48, 48),
    color_mode='grayscale',
    batch_size=32,
    class_mode='categorical',
    seed=seed_value,
    shuffle=False # Viktigt för korrekt utvärdering
)
```

Nu när vi har mappstrukturen för tränings- och testdata, där varje undermapp representerar en emotion (som "angry", "happy", "surprise" etc.), kan vi använda en ImageDataGenerator för att ladda bilderna direkt från mapparna och träna modellen.

Eftersom varje undermapp motsvarar en klass, kan vi använda ***flow_from_directory***-funktionen för att ladda och förbereda datasetet.

Validation split: Nu har vi lagt till validation_split=0.2 i datageneratoren, vilket betyder att 20% av träningsdatan delas upp för validering.

Träningsdata: använder de återstående 80% av träningsdatan för att faktiskt träna modellen.

Valideringsdata: använder de 20% av träningsdatan som är reserverade för validering under träningsprocessen.

Testdata: Detta laddas separat från mappen ./test och används efter modellträning för slutgiltig utvärdering.

CNN modellen

```
from tensorflow.keras import layers, models

# Bygg CNN-modellen
model = models.Sequential()

# Första konvolutionella lagret
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(48, 48, 1)))
model.add(layers.MaxPooling2D((2, 2)))

# Andra konvolutionella lagret
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Tredje konvolutionella lagret
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Flattening för fullt anslutet nätverk
model.add(layers.Flatten())

# Fullt anslutna lager
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dropout(0.5)) # Dropout för att minska överanpassning

# Utgångslagret
model.add(layers.Dense(7, activation='softmax')) # Eftersom vi har 7 klasser

# Kompilera modellen
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

```
# Räkna ut class weights från train_generator
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_generator.classes),
    y=train_generator.classes
)

class_weight_dict = dict(enumerate(class_weights))
print("Class weights:", class_weight_dict)

# Valfritt: visa etiketter kopplade till vikter
label_map = train_generator.class_indices
inv_label_map = {v: k for k, v in label_map.items()}
for class_index, weight in class_weight_dict.items():
    print(f"Klass '{inv_label_map[class_index]}' får vikt: {weight:.2f}")
```

Träna modellen

```
# Träna modellen
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size,
    epochs=50,
    class_weight=class_weight_dict)
```

history = model.fit(...):

model.fit används för att träna modellen på träningsdata.

Variabeln history lagrar träningshistoriken, som innehåller information om tränings- och valideringsförlust, noggrannhet och andra mätvärden vid varje epoch.

train_generator:

Detta är generatoren som producerar träningsdatan. Eftersom data laddas i batcher, används generatoren för att strömma data i små portioner från träningsdatasetet.

Generatoren skapas av ImageDataGenerator och flow_from_directory, vilket gör att stora dataset kan laddas in stegvis utan att behöva hålla allt i minnet.

steps_per_epoch=train_generator.samples // train_generator.batch_size:

steps_per_epoch definierar hur många batcher som ska användas per epoch.

Det beräknas genom att dividera det totala antalet träningsbilder (train_generator.samples) med batchstorleken (train_generator.batch_size).

Detta innebär att nätverket tränas på alla bilder i datasetet under en epoch. Varje batch innehåller en viss del av träningsbilderna, och när alla batcher har körts, anses en epoch vara klar.

validation_data=validation_generator:

Valideringsdata används för att utvärdera modellens prestanda på data som den inte har tränats på, vid slutet av varje epoch.

Här används validation_generator, som innehåller valideringsbilderna och deras etiketter.

validation_steps=validation_generator.samples // validation_generator.batch_size:

validation_steps definierar hur många batcher som ska användas för att utvärdera valideringsdatan.

Det beräknas på samma sätt som för träningsdata genom att dividera antalet valideringsbilder med batchstorleken. Detta gör att alla valideringsbilder används vid slutet av varje epoch för att mäta modellens prestanda.

epochs=50:

epochs anger hur många gånger hela träningsdatasetet ska bearbetas (i form av batcher).

I detta fall är värdet 50, vilket innebär att modellen kommer att gå igenom hela träningsdatasetet 50 gånger.

TF 2.19.0: WSL CNN modellen

```
from tensorflow.keras import layers, models

# CNN-modellen för FER-2013
model = models.Sequential([
    layers.Input(shape=(48, 48, 1)), # Tydligt input-lager

    # Första konvolutionella blocket
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Andra konvolutionella blocket
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Tredje konvolutionella blocket
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Flattening
    layers.Flatten(),

    # Fullt anslutna lager
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5), # Minskar överanpassning

    # Utgångslager (7 klasser för FER-2013)
    layers.Dense(7, activation='softmax')
])

# Kompilera modellen
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Visa sammanfattning av modellen
model.summary()
```

```
# Räkna ut class weights från train_generator (inte "reduced")
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_generator.classes),
    y=train_generator.classes
)

class_weight_dict = dict(enumerate(class_weights))
print("Class weights:", class_weight_dict)

label_map = train_generator.class_indices
inv_label_map = {v: k for k, v in label_map.items()}

for class_index, weight in class_weight_dict.items():
    print(f"Klass '{inv_label_map[class_index]}' får vikt: {weight:.2f}")
```

```
# Träna med class_weight
history = model.fit(
    train_generator,
    epochs=50,
    validation_data=validation_generator,
    class_weight=class_weight_dict
)
```


Utvärdera modellen på testdatan

```
# Utvärdera modellen på testdatan
test_loss, test_acc = model.evaluate(test_generator, steps=test_generator.samples // test_generator.batch_size)
print(f'Test accuracy: {test_acc}')
```

```
224/224 [=====] - 2s 9ms/step - loss: 2.7883 - accuracy: 0.5550
Test accuracy: 0.5549665093421936
```

model.evaluate():

Denna metod används för att utvärdera den tränade modellen på en ny dataset (i detta fall testdatan) och returnerar två värden:

test_loss: Förlusten (loss) på testdatan, vilket mäter hur bra (eller dåligt) modellen presterar.

test_acc: Noggrannheten (accuracy) på testdatan, vilket anger andelen korrekta förutsägelser modellen gjorde på testdatan.

test_generator:

Detta är testgeneratoren som laddar och förbereder testdatan (i batchar) för utvärdering. Det fungerar på samma sätt som träningsgeneratoren, men det används för att utvärdera modellen.

steps=test_generator.samples // test_generator.batch_size:

Här specificeras hur många steg som behövs för att bearbeta hela testdatasetet under utvärderingen.

test_generator.samples: Det totala antalet bilder i testdatan.

test_generator.batch_size: Hur många bilder som laddas i varje batch.

// (heltalsdivision): används // för att dela antalet testbilder med batchstorleken och få ett heltal (antal batcher som behövs för att bearbeta hela datasetet).

Accuracy 0.55 är det bra eller dålig?

Antalet klasser (multiklassklassificering vs. binär klassificering):

- FER-2013 datasetet har sju olika klasser (angry, disgust, fear, happy, neutral, sad, surprise), vilket gör det till ett multiklassklassificeringsproblem.
- För ett sådant problem är det generellt svårare för en modell att göra rätt förutsägelser jämfört med binär klassificering (som bara har två klasser).
- Om det vore en binär klassificering, skulle vi förvänta oss en högre noggrannhet, men eftersom vi har fler klasser, kan en noggrannhet på 0.55 indikera att modellen har viss förmåga att korrekt klassificera bilder, men det finns också mycket utrymme för förbättring.

Jämförelse med en slumpmässig gissning:

- Om vi skulle göra slumpmässiga gissningar i ett problem med sju klasser, skulle du ha en genomsnittlig chans på $1/7$ (ca 14% accuracy).
- I det här fallet, med en accuracy på 55%, är modellen betydligt bättre än en slumpmässig gissning, men den har fortfarande en hel del felaktiga förutsägelser.

Komplexitet i problemet:

- Ansiktsuttrycksigenkänning är ett utmanande problem på grund av variationer i ansiktsuttryck, belysning, vinklar och andra faktorer.
- Det är normalt att noggrannheten initialt kan vara låg, särskilt om modellen inte är finjusterad eller om datasetet är komplext.

Förbättringspotential:

- Att finjustera hyperparametrarna (antal filter, antal lager, lärhastighet).
- Att använda data augmentation för att göra modellen mer robust mot olika varianter av indata (rotationer, förskjutningar, ljusförhållanden).
- Att träna modellen under fler epoker.

Utmaning i emotionell klassificering:

- Emotionell klassificering kan vara svårt eftersom uttryck kan variera mycket mellan individer, och ibland är gränserna mellan olika känslor (t.ex. "fear" och "surprise") svåra att skilja.
- En noggrannhet på 0.56 kan vara ett bra utgångspunkt för att fortsätta förbättra modellen genom ytterligare experimentering.

Confusion Matrix

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

# Gör förutsägelser på testdatan
predictions = model.predict(test_generator)

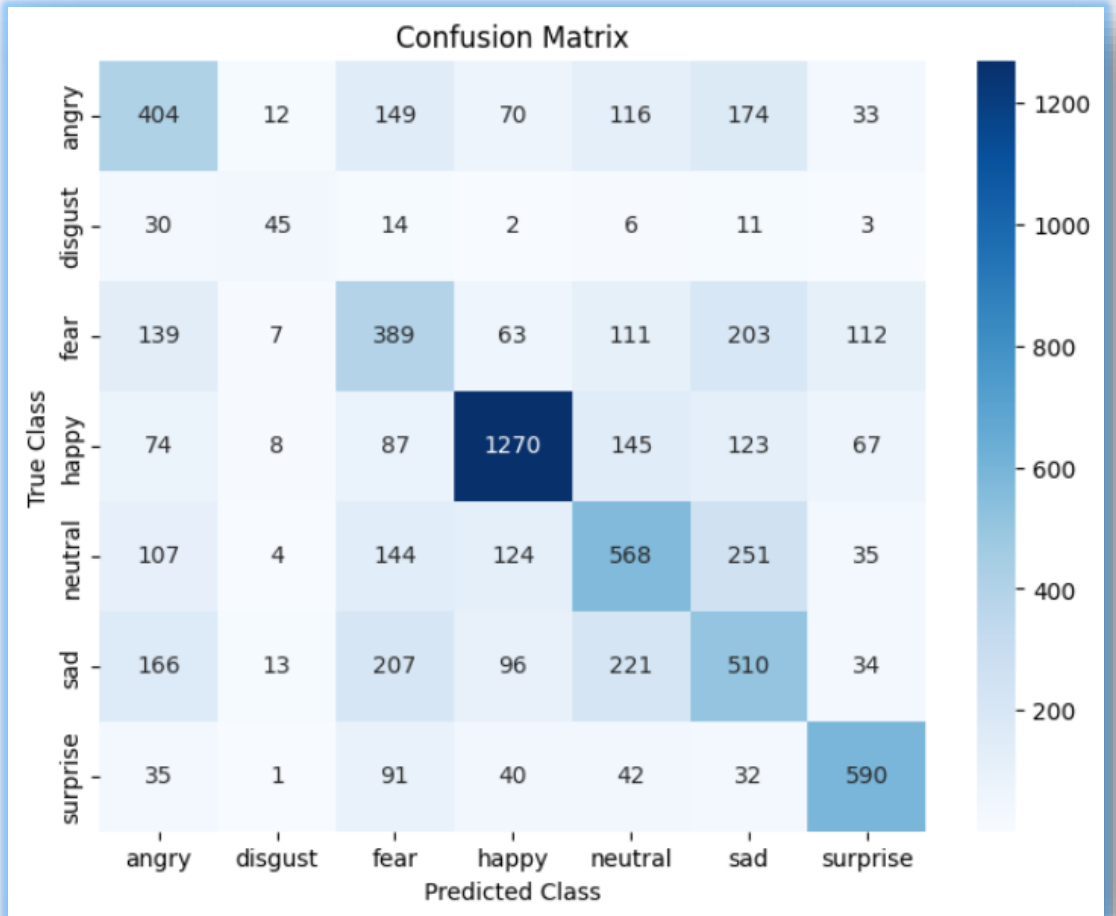
# Omvandla förutsägelserna till klassetiketter (argmax för att få den högsta sannolikheten)
y_pred_classes = np.argmax(predictions, axis=1)

# Sanna etiketter från testgeneratoren
y_true = test_generator.classes

# Skapa confusion matrix igen
conf_matrix = confusion_matrix(y_true, y_pred_classes)

# Skapa en heatmap för confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues",
            xticklabels=train_generator.class_indices.keys(),
            yticklabels=train_generator.class_indices.keys())
plt.xlabel('Predicted Class')
plt.ylabel('True Class')
plt.title('Confusion Matrix')
plt.show()

# Visa felklassificeringar
for i in range(len(conf_matrix)):
    for j in range(len(conf_matrix)):
        if i != j and conf_matrix[i][j] > 0:
            print(f"Class {i} misclassified as Class {j}: {conf_matrix[i][j]} instances")
```



Confusion matrix get info inte bara om modellens prestanda men också hur olika klasser blandas. Kanske visa klasser vi kan ta bort om dem är mindre viktiga för oss, visa kanske vi skall granska och markera om bilder

Classification report

```
from sklearn.metrics import classification_report
import numpy as np

# Gör förutsägelser på testdatan (det här ger sannolikheter från softmax-lagret)
y_pred_prob = model.predict(test_generator)

# Omvandla sannolikheterna till klassprediktioner (argmax tar den klass med högsta sannolikheten)
y_pred_classes = np.argmax(y_pred_prob, axis=1)

# Eftersom test_generator används måste vi hämta de sanna etiketterna från generatoren
y_true = test_generator.classes # Sanna etiketter från testdata

# Skapa och skriv ut classification report
report = classification_report(y_true, y_pred_classes, target_names=test_generator.class_indices.keys())
print(report)
```

	precision	recall	f1-score	support
angry	0.42	0.42	0.42	958
disgust	0.50	0.41	0.45	111
fear	0.36	0.38	0.37	1024
happy	0.76	0.72	0.74	1774
neutral	0.47	0.46	0.47	1233
sad	0.39	0.41	0.40	1247
surprise	0.68	0.71	0.69	831
accuracy			0.53	7178
macro avg	0.51	0.50	0.51	7178
weighted avg	0.53	0.53	0.53	7178

Träning och Validering: Noggrannhet/ Accuracy

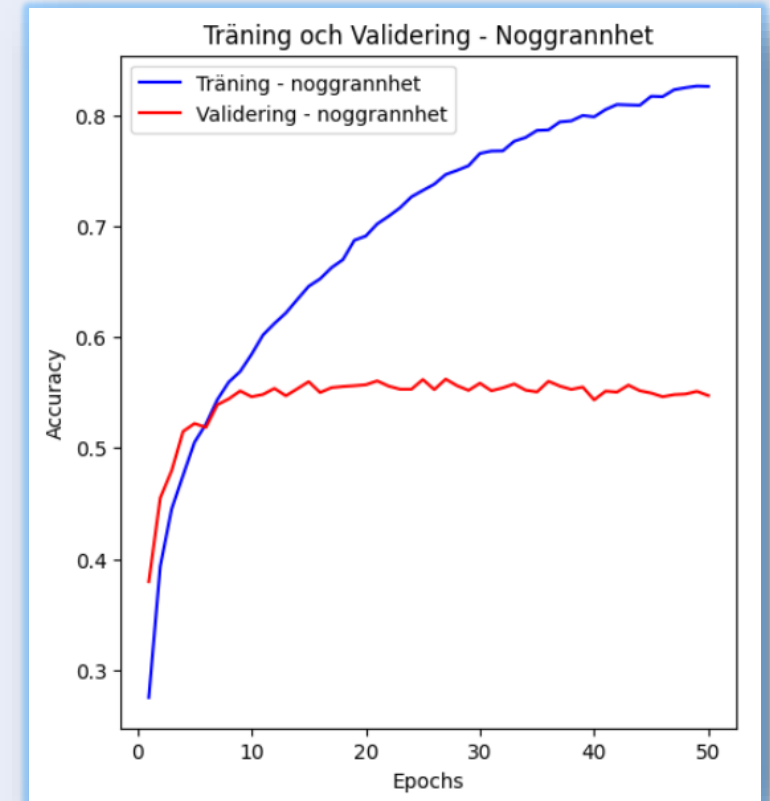
```
import matplotlib.pyplot as plt

# Hämta värden från träningshistoriken
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

# Plotta tränings- och valideringsnoggrannhet
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.plot(epochs, acc, 'b', label='Träning - noggrannhet')
plt.plot(epochs, val_acc, 'r', label='Validering - noggrannhet')
plt.title('Träning och Validering - Noggrannhet')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```



Träningsnoggrannhet (blå linje) fortsätter att öka över tid och når nästan 0.9.

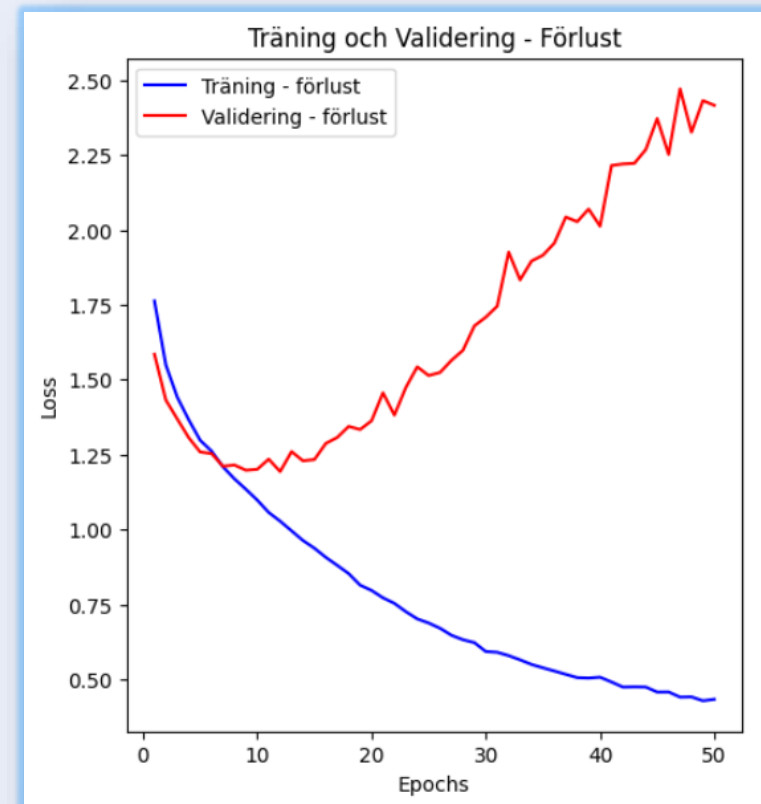
Valideringsnoggrannhet (röd linje) når en topp vid ungefär 0.56 efter cirka 10–15 epochs och planar sedan ut eller varierar något utan att förbättras.

Detta indikerar att modellen lär sig mycket bra på träningsdatan, men dess generaliseringsförmåga till ny data (valideringsdatan) är begränsad. Valideringsnoggrannheten förbättras inte efter en viss punkt, vilket tyder på att modellen överanpassar träningsdatan.

Träning och Validering: Förlust/ Loss

```
# Plotta tränings- och valideringsförlust
plt.subplot(1, 2, 2)
plt.plot(epochs, loss, 'b', label='Träning - förlust')
plt.plot(epochs, val_loss, 'r', label='Validering - förlust')
plt.title('Träning och Validering - Förlust')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```



Träningsförlust (blå linje) minskar stadigt och når mycket låga värden, vilket visar att modellen lyckas minimera förlusten på träningsdatan.

Valideringsförlust (röd linje) minskar initialt, men börjar sedan öka igen efter runt 10 epochs.

Detta är ett tydligt tecken på **överanpassning**, då modellen börjar lära sig specifika mönster från träningsdatan som inte generaliserar till valideringsdatan.

Åtgärder för att förbättra modellens prestanda

Early Stopping:

Använd en "Early Stopping" -callback för att sluta träna modellen när valideringsförlusten börjar öka. Detta kan förhindra överanpassning genom att stoppa träningen när prestandan på valideringsdatan börjar försämrast.

Regulärisering:

Lägg till L2-regulärisering eller Dropout i modellens dolda lager för att minska överanpassning. Detta kommer att straffa stora vikter och därmed göra modellen mer robust mot överanpassning.

Data Augmentation:

Förbättra generaliseringsförmågan genom att använda data augmentation på träningsdatan. Detta innebär att skapa nya träningsbilder genom att rotera, skala, eller skifta befintliga bilder, vilket gör att modellen ser fler varianter av datan.

Data Augmentation

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

seed_value = 42

# Data augmentation för träning och validering
train_val_datagen = ImageDataGenerator(
    rescale=1.0/255.0,
    validation_split=0.2,
    rotation_range=8,
    width_shift_range=0.08,
    height_shift_range=0.08,
    shear_range=0.05,
    zoom_range=0.08,
    horizontal_flip=True,
    fill_mode='nearest'
)

# Test utan augmentation
test_datagen = ImageDataGenerator(rescale=1.0/255.0)

# Träningsgenerator med data augmentation
train_generator = train_val_datagen.flow_from_directory(
    './train',
    target_size=(48, 48),
    color_mode='grayscale',
    batch_size=32,
    class_mode='categorical',
    subset='training',
    seed=seed_value
)

# Valideringsgenerator med data augmentation
validation_generator = train_val_datagen.flow_from_directory(
    './train',
    target_size=(48, 48),
    color_mode='grayscale',
    batch_size=32,
    class_mode='categorical',
    subset='validation',
    seed=seed_value
)

# Testgenerator utan augmentation
test_generator = test_datagen.flow_from_directory(
    './test',
    target_size=(48, 48),
    color_mode='grayscale',
    batch_size=32,
    class_mode='categorical',
    seed=seed_value,
    shuffle=False # Viktigt för korrekt utvärdering
)
```

Augmentering sker "on the fly" – i minnet

- Bildfilerna på disken ändras inte.
- Varje gång en bild läses in till ett träningsbatch (med `.flow_from_directory(...)`), skapas en ny version av bilden.

Augmentering är slumpmässig (random)

- Varje augmenteringsparameter fungerar som ett intervall (t.ex. rotation mellan -10° och $+10^\circ$).
- För varje bild och varje epoch, tillämpas slumpmässiga variationer inom de angivna gränserna.
- Det innebär att samma bild kan se olika ut varje gång den används i träningen.

CNN Modellen: Dropout

```
# Kompilera och bygga CNN-modellen med Dropout och EarlyStopping
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping

# Bygg CNN-modellen
model = models.Sequential()

# Första konvolutionella lagret
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(48, 48, 1)))
model.add(layers.MaxPooling2D((2, 2)))

# Andra konvolutionella lagret
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Tredje konvolutionella lagret
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Flatten-lager
model.add(layers.Flatten())

# Fullt anslutna lager
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dropout(0.5)) # Dropout för att minska överanpassning

# Lägg till ytterligare ett fullt anslutet lager med Dropout
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dropout(0.3))

# Utgångslagret
model.add(layers.Dense(7, activation='softmax'))

# Kompilera modellen
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

Dropout

Early Stopping

```
# Räkna ut class weights från train_generator
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_generator.classes),
    y=train_generator.classes
)

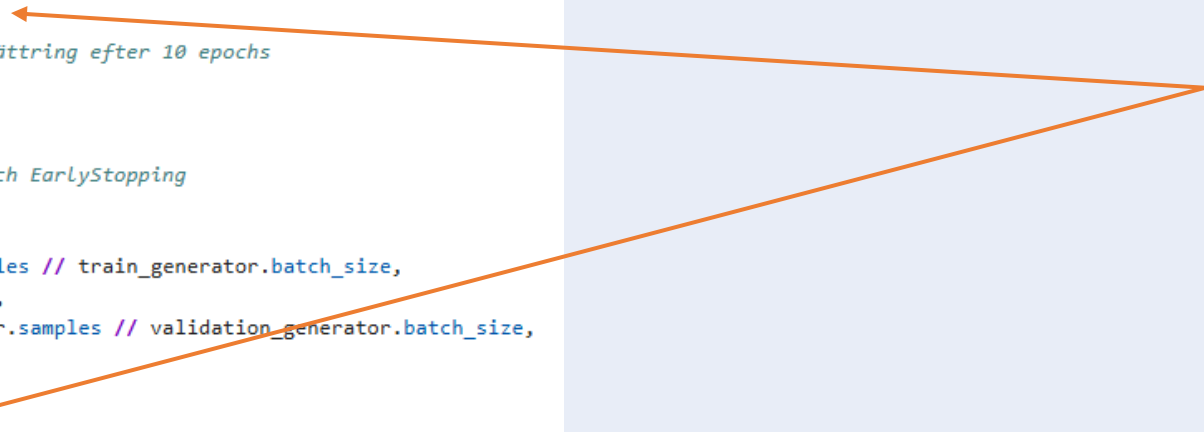
class_weight_dict = dict(enumerate(class_weights))
print("Class weights:", class_weight_dict)

# Valfritt: visa etiketter kopplade till vikter
label_map = train_generator.class_indices
inv_label_map = {v: k for k, v in label_map.items()}
for class_index, weight in class_weight_dict.items():
    print(f"Klass '{inv_label_map[class_index]}' får vikt: {weight:.2f}")

# Skapa en EarlyStopping callback
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=10, # Stoppa om ingen förbättring efter 10 epochs
    restore_best_weights=True
)

# Träna modellen med data augmentation och EarlyStopping
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size,
    class_weight=class_weight_dict,
    epochs=50, # Max antal epochs
    callbacks=[early_stopping]
)
```

Early Stopping



TF 2.19.0: WSL CNN modellen

```
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping

# Bygg CNN-modellen med tydligt Input-lager
model = models.Sequential([
    layers.Input(shape=(48, 48, 1)), # Tydligt input-lager

    # Första konvolutionella blocket
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Andra konvolutionella blocket
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Tredje konvolutionella blocket
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Flattening-lager
    layers.Flatten(),

    # Fullt anslutet lager med Dropout
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5), # Minskar överanpassning

    # Ytterligare fullt anslutet lager med Dropout
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),

    # Utgångslager (7 klasser för FER-2013)
    layers.Dense(7, activation='softmax')
])
```

```
# Kompilera modellen
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Sammanfattning av modellen (för verifiering)
model.summary()

# EarlyStopping-callback för att stoppa vid överanpassning
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

# Räkna ut class weights från train_generator
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_generator.classes),
    y=train_generator.classes
)

class_weight_dict = dict(enumerate(class_weights))
print("Class weights:", class_weight_dict)

label_map = train_generator.class_indices
inv_label_map = {v: k for k, v in label_map.items()}

for class_index, weight in class_weight_dict.items():
    print(f"Klass '{inv_label_map[class_index]}' får vikt: {weight:.2f}")

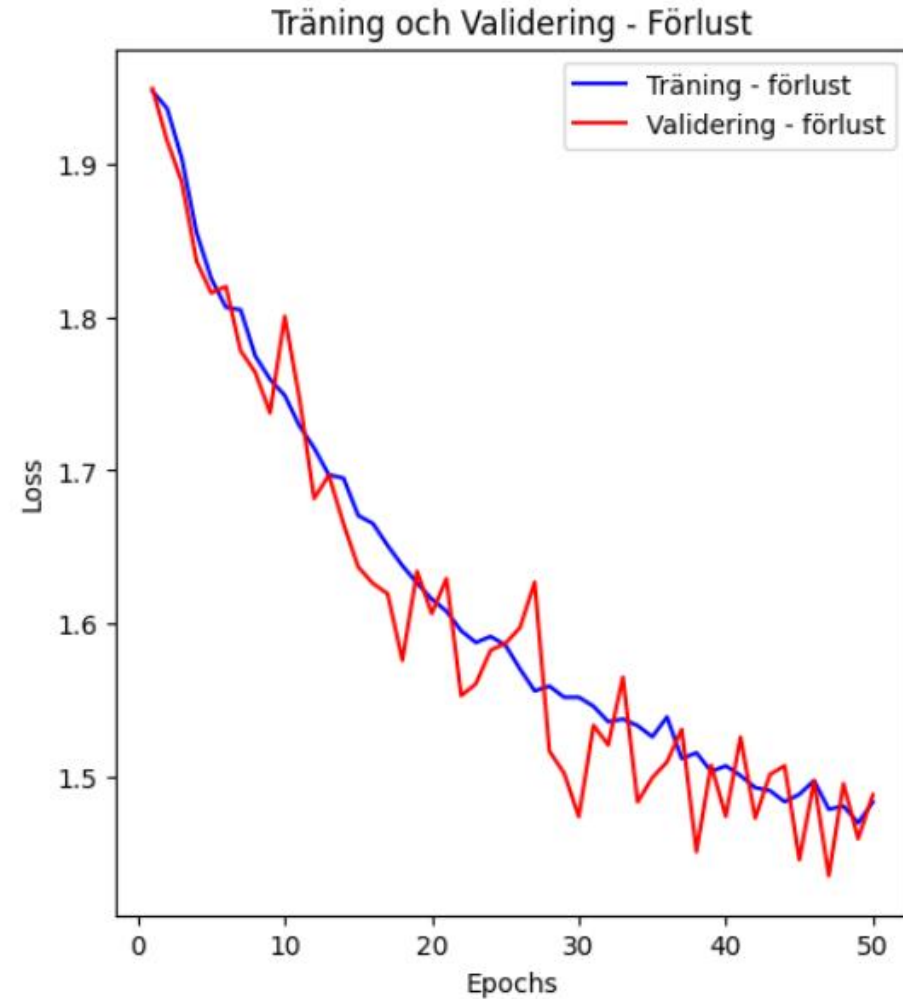
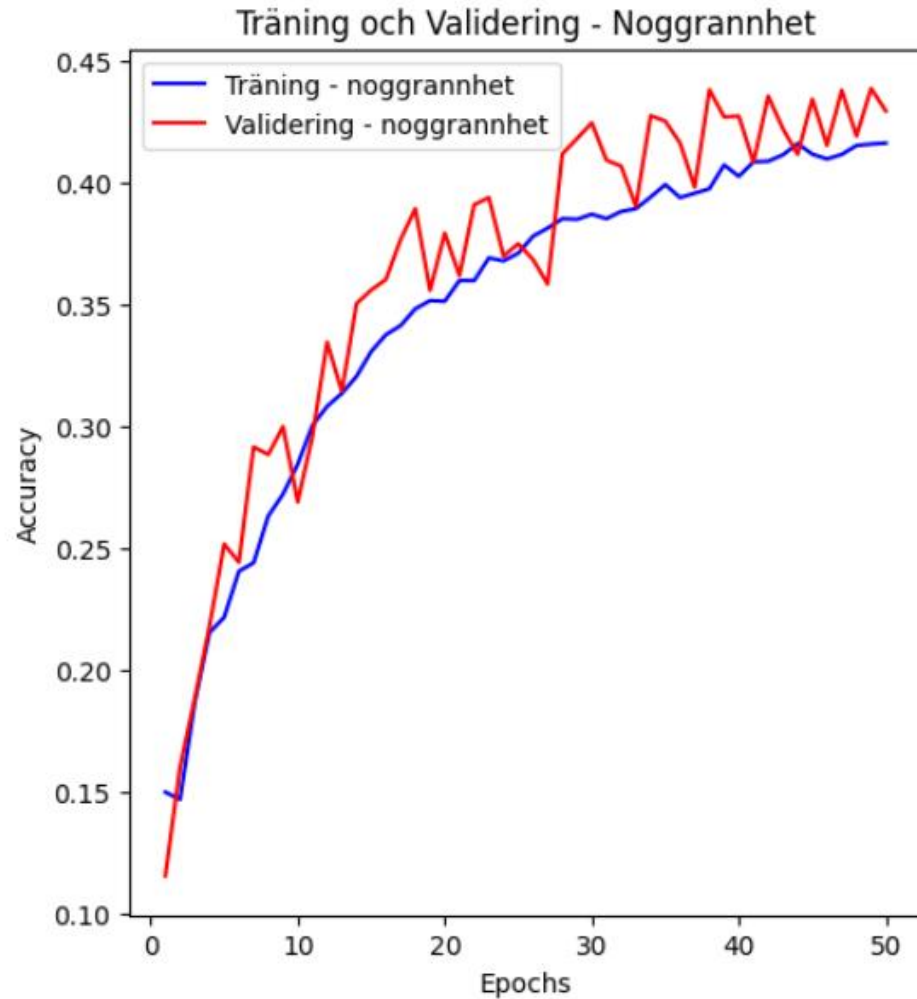
# Träna modellen med data augmentation och EarlyStopping
history = model.fit(
    train_generator,
    validation_data=validation_generator,
    class_weight=class_weight_dict,
    epochs=50,
    callbacks=[early_stopping]
)
```

Modell Utvärdering

```
# Utvärdera modellen på testdatan  
test_loss, test_acc = model.evaluate(test_generator, steps=test_generator.samples // test_generator.batch_size)  
print(f'Test accuracy: {test_acc}')
```

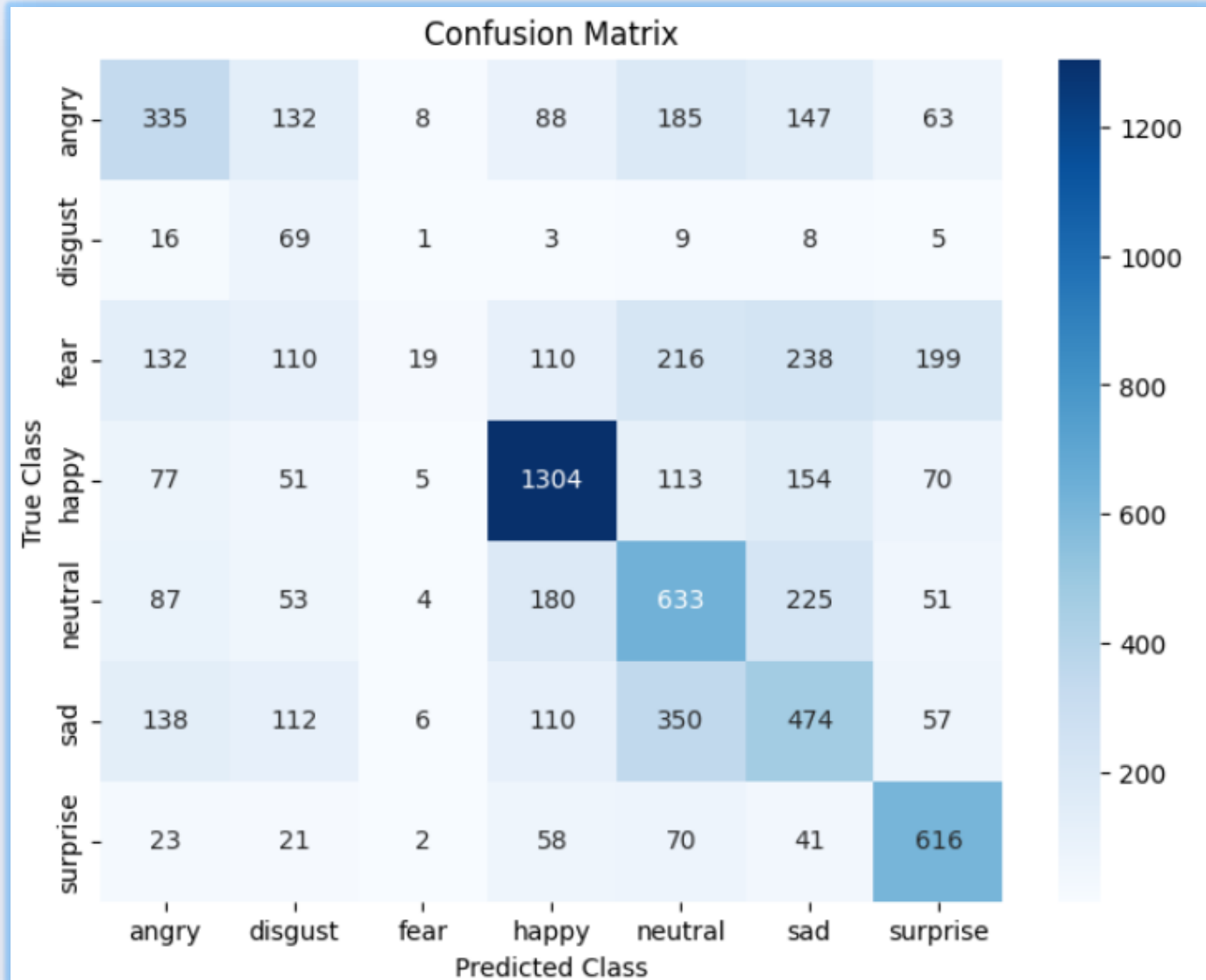
```
Test accuracy: 0.4799107015132904
```

Träning och Validering



Kanske vi kan köra flera epoch?

Confusion Matrix



Classification Report

	precision	recall	f1-score	support
angry	0.41	0.35	0.38	958
disgust	0.13	0.62	0.21	111
fear	0.42	0.02	0.04	1024
happy	0.70	0.74	0.72	1774
neutral	0.40	0.51	0.45	1233
sad	0.37	0.38	0.37	1247
surprise	0.58	0.74	0.65	831
accuracy			0.48	7178
macro avg	0.43	0.48	0.40	7178
weighted avg	0.49	0.48	0.45	7178

Spara modellen

```
# Spara i SavedModel format  
model.save('saved_model/emotion_recognition_model')
```

```
from tensorflow.keras.models import load_model  
  
# Ladda tillbaka den sparade modellen  
model = load_model('saved_model/emotion_recognition_model')
```

TF 2.19.0

```
# Spara modellen i Keras-format (.keras)  
model.save('fer2013_augmented_model.keras')
```

```
from tensorflow.keras.models import load_model  
  
# Ladda modellen (Keras-format)  
loaded_model = models.load_model('fer2013_augmented_model.keras')
```


Vidareutveckling

För att förbättra resultaten kan vi överväga att:

- ❖ Använda data augmentation för att öka antalet exempel i svagare klasser.
- ❖ Justera modellen eller prova mer avancerade nätverksarkitekturer.
- ❖ Använda mer data om möjligt för att få bättre träningsresultat.
- ❖ Kolla genom data och förfina kategorier eller slå ihop vissa kategorier (Fear och Surprise; Disgust och Angry; Sad och Neutral) som överlappar med varandra.

Färre kategorier:

- Positiva känslor: Happy
- Neutrala känslor: Neutral
- Negativa känslor: Angry, Fear, Sad, Disgust

Vidareutveckling: Kolla med UMAP

```
import umap.umap_ as umap
import matplotlib.pyplot as plt
import seaborn as sns

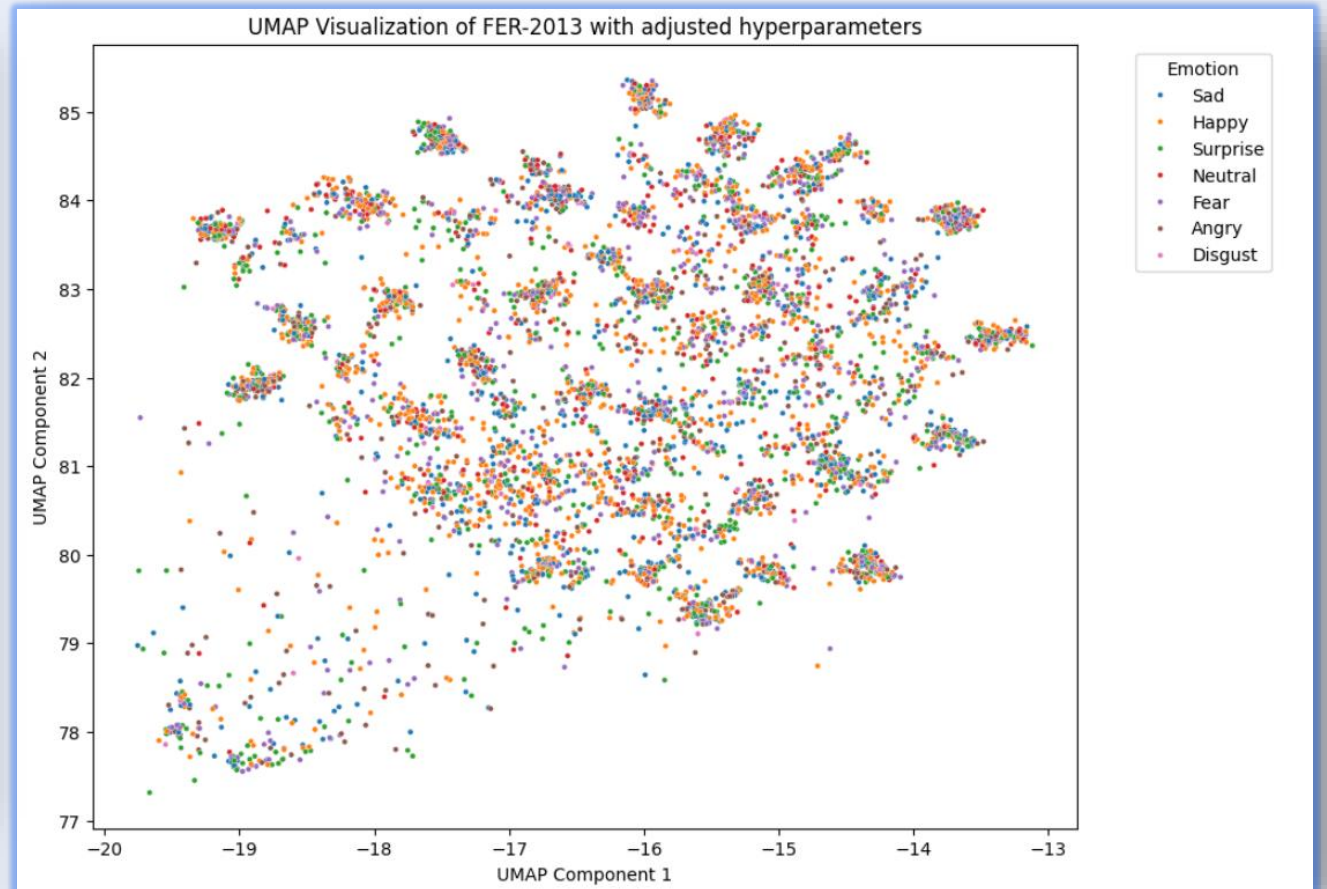
# Ändra UMAP-inställningar för att tydligare se data-mönster
reducer = umap.UMAP(
    n_neighbors=150,
    min_dist=0.05,      # försök olika värden mellan 0.01 och 0.5
    n_components=2,
    metric='dice',
    random_state=42
)

embedding = reducer.fit_transform(X_val_flat)

emotion_labels = ['Angry', 'Disgust', 'Fear', 'Happy', 'Sad', 'Surprise', 'Neutral']

plt.figure(figsize=(10, 8))
sns.scatterplot(x=embedding[:, 0], y=embedding[:, 1],
               hue=[emotion_labels[label] for label in y_labels],
               palette='tab10', s=10)

plt.title('UMAP Visualization of FER-2013 with adjusted hyperparameters')
plt.xlabel('UMAP Component 1')
plt.ylabel('UMAP Component 2')
plt.legend(title='Emotion', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```



Vidareutveckling: Färre kategorier:

re-arrangera bilder: TF 2.19.0

```
import os
import shutil

original_to_new_labels = {
    'happy': 'Positive',
    'neutral': 'Neutral',
    'angry': 'Negative',
    'fear': 'Negative',
    'sad': 'Negative',
    'disgust': 'Negative',
    'surprise': 'Neutral' # eller 'Positive', baserat på din preferens
}

def reorganize_data(src_folder, dst_folder, mapping):
    if not os.path.exists(dst_folder):
        os.makedirs(dst_folder)

    for old_label, new_label in mapping.items():
        old_path = os.path.join(src_folder, old_label)
        new_path = os.path.join(dst_folder, new_label)

        if not os.path.exists(new_path):
            os.makedirs(new_path)

        for filename in os.listdir(old_path):
            shutil.copy(os.path.join(old_path, filename), new_path)

# Exempel för att köra:
reorganize_data('./train', './train_reduced', original_to_new_labels)
reorganize_data('./test', './test_reduced', original_to_new_labels)
```

Vidareutveckling: Färre kategorier:

modell, TF 2.19.0

```
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

train_val_datagen = ImageDataGenerator(
    rescale=1.0/255.0,
    validation_split=0.2,
    rotation_range=8,
    width_shift_range=0.08,
    height_shift_range=0.08,
    shear_range=0.05,
    zoom_range=0.08,
    horizontal_flip=True,
    fill_mode='nearest'
)

train_generator_reduced = train_val_datagen.flow_from_directory(
    './train_reduced',
    target_size=(48, 48),
    color_mode='grayscale',
    batch_size=32,
    class_mode='categorical',
    subset='training',
    seed=42
)

validation_generator_reduced = train_val_datagen.flow_from_directory(
    './train_reduced',
    target_size=(48, 48),
    color_mode='grayscale',
    batch_size=32,
    class_mode='categorical',
    subset='validation',
    seed=42
)

test_datagen = ImageDataGenerator(rescale=1.0/255.0)

test_generator_reduced = test_datagen.flow_from_directory(
    './test_reduced',
    target_size=(48, 48),
    color_mode='grayscale',
    batch_size=32,
    class_mode='categorical',
    shuffle=False,    # Viktigt! För korrekt jämförelse
    seed=42
)
```

```
# -----
# Beräkna class weights
# -----
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_generator_reduced.classes),
    y=train_generator_reduced.classes
)
class_weight_dict = dict(enumerate(class_weights))
print("Class weights:", class_weight_dict)

label_map = train_generator_reduced.class_indices
inv_map = {v: k for k, v in label_map.items()}
for i, w in class_weight_dict.items():
    print(f"Klass '{inv_map[i]}' : vikt {w:.2f}")

# -----
# Bygg en lättare men effektiv CNN
# -----
model = models.Sequential([
    layers.Input(shape=(48, 48, 1)),

    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(3, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

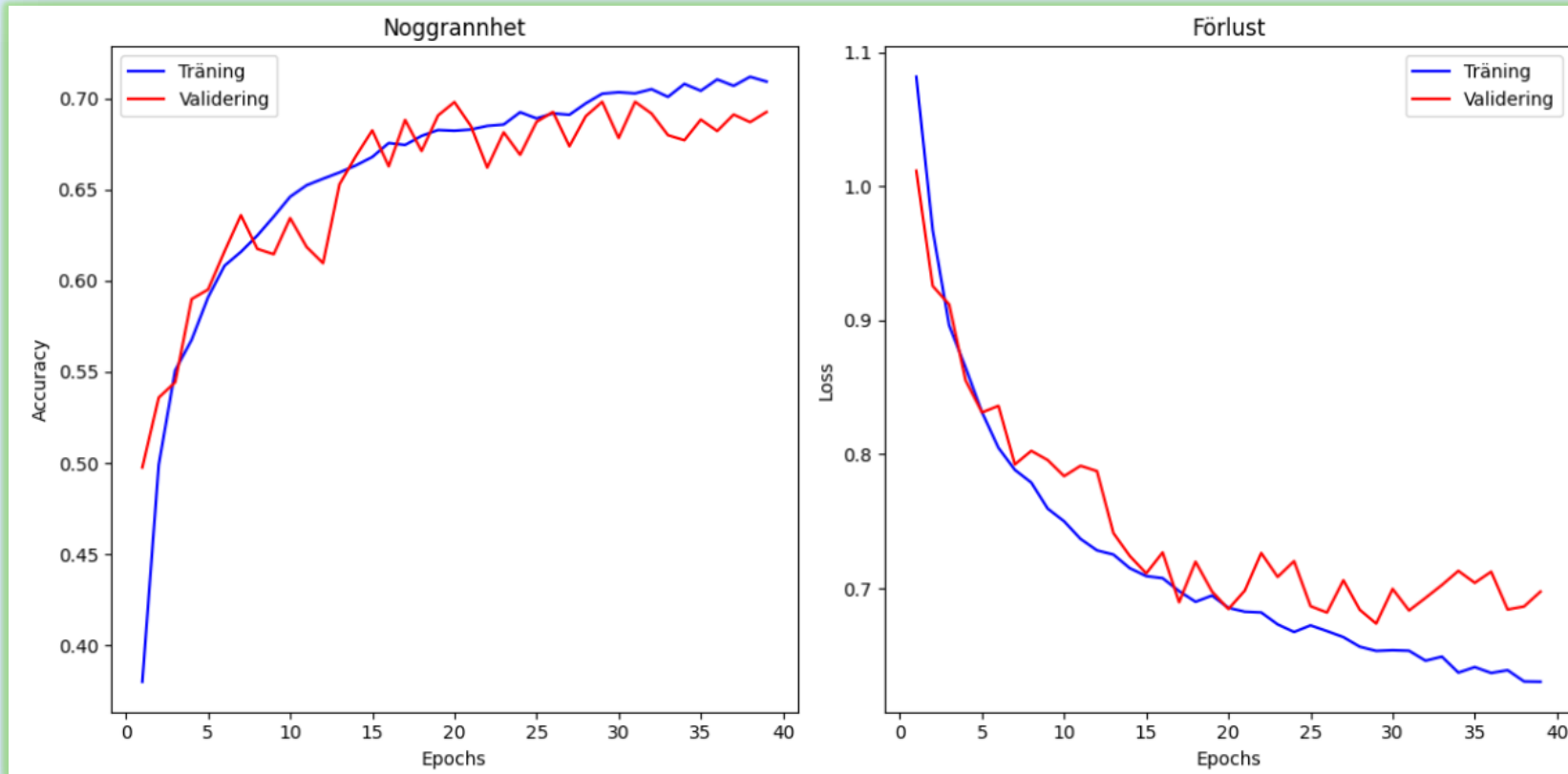
model.summary()
```

```
# -----
# Early stopping
# -----
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

# -----
# Träna modellen med klasssvikter
# -----
history = model.fit(
    train_generator_reduced,
    validation_data=validation_generator_reduced,
    epochs=50,
    callbacks=[early_stopping],
    class_weight=class_weight_dict
)
```

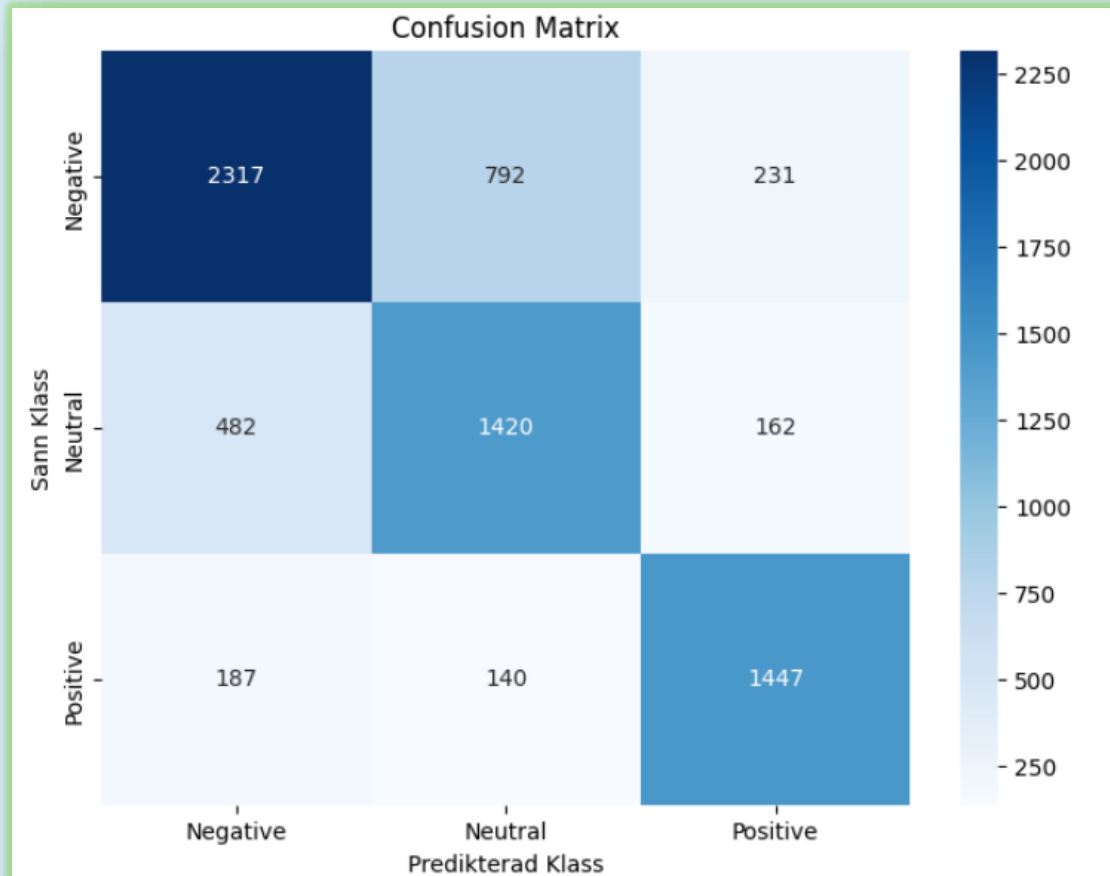
Vidareutveckling: Färre kategorier:

Modell utvärdering, TF 2.19.0



Vidareutveckling: Färre kategorier:

Modell utvärdering, TF 2.19.0



Classification Report:				
	precision	recall	f1-score	support
Negative	0.78	0.69	0.73	3340
Neutral	0.60	0.69	0.64	2064
Positive	0.79	0.82	0.80	1774
accuracy			0.72	7178
macro avg	0.72	0.73	0.73	7178
weighted avg	0.73	0.72	0.72	7178

Små projekt: CNN



- **Välj ett valfritt dataset med bilder, här är några exempel:**
 - CIFAR-10,
 - Fashion-MNIST,
 - Caltech-101,
 - CelebA (CelebFaces Attributes Dataset)
 - Tiny ImageNet
 - Kaggle Cats vs Dogs
- **Bygg CNN Neural Nätverks modellen**
- **Utvärdera modellen**
- **Behöver den justeras?**
- **Presentation:**
 - Beskriv datasetet.
 - Presentera utveckling och resultat av CNN modellen.
 - Kan modellen förbättras? Vilka problem finns? Vilka lösningar är tänkbara?
 - Presentera resultat som databerättelse (runt 15 min). Gör en story.

Tack för er tid